

Dissertation presented to the Instituto Tecnológico de Aeronáutica, in partial fulfillment of the requirements for the degree of Master of Science in the Graduate Program of Aeronautical Engineering, Field of Aeronautical Systems.

Lucas Ávila Silva

IMAGE-BASED GIMBAL CONTROL USING A CONVOLUTIONAL NEURAL NETWORK

Dissertation approved in its final version by signatories below:

D.Sc. Marcos R. Omena de A. Máximo

Advisor

M.Sc. Raphael das Neves Calvo

Co-advisor

Campo Montenegro
São José dos Campos, SP - Brazil
2025

Cataloging-in Publication Data
Documentation and Information Division

Silva, Lucas Ávila

Image-Based Gimbal Control Using a Convolutional Neural Network / Lucas Ávila Silva.
São José dos Campos, 2025.
73f.

Dissertation of Master of Science – Course of Aeronautical Engineering. Area of Aeronautical Systems – Instituto Tecnológico de Aeronáutica, 2025. Advisor: D.Sc. Marcos R. Omena de A. Máximo. Co-advisor: M.Sc. Raphael das Neves Calvo.

I. Instituto Tecnológico de Aeronáutica. II. Title.

BIBLIOGRAPHIC REFERENCE

SILVA, Lucas Ávila. **Image-Based Gimbal Control Using a Convolutional Neural Network**. 2025. 73f. Dissertation of Master of Science – Instituto Tecnológico de Aeronáutica, São José dos Campos.

CESSION OF RIGHTS

AUTHOR'S NAME: Lucas Ávila Silva

PUBLICATION TITLE: Image-Based Gimbal Control Using a Convolutional Neural Network.

PUBLICATION KIND/YEAR: Dissertation / 2025

It is granted to Instituto Tecnológico de Aeronáutica permission to reproduce copies of this dissertation and to only loan or to sell copies for academic and scientific purposes. The author reserves other publication rights and no part of this dissertation can be reproduced without the authorization of the author.

Lucas Ávila Silva

Av. Juscelino Kubitschek, 5460, apt. 134
12.215-380 – São José dos Campos–SP

IMAGE-BASED GIMBAL CONTROL USING A CONVOLUTIONAL NEURAL NETWORK

Lucas Ávila Silva

Thesis Committee Composition:

D.Sc.	Carlos Forster	Member	-	ITA
D.Sc.	Marcos R. Omena de A. Máximo	Advisor	-	ITA
M.Sc.	Raphael das Neves Calvo	Co-advisor	-	EMBRAER
D.Sc.	Alessandro Jacoud Peixoto	Member	-	UFRJ

This work is dedicated, first and foremost, to God. My deepest gratitude goes to all who, with their love, patience, and wisdom, made this dream a reality. To my beloved family, especially my parents, for their unwavering support, the profound values they instilled, and the sacrifices made so I could embark on this journey. Your love and faith were my most powerful inspiration. And to my dear fiancée, for her steadfast companionship, patience, and constant encouragement. Your love and understanding provided refuge and strength during my most challenging moments.

Acknowledgments

I extend my deepest gratitude to my advisor, Professor D.Sc. Marcos R. Omena de A. Máximo, and co-advisor, Professor M.Sc. Raphael das Neves Calvo, for their invaluable guidance. My sincere thanks also go to Professors D.Sc. Carlos Forster and D.Sc. Alessandro Jacoud Peixoto for their insightful contributions as examining board members. To my beloved family and dear fiancée, your unwavering love and support were my constant motivation. This achievement is dedicated to all of you.

"Attitude is a little thing that makes a big difference."
— WINSTON CHURCHILL

Resumo

A detecção de objetos em tempo real é fundamental para o avanço de sistemas autônomos e de segurança. A integração dessa tecnologia a gimbals multi-eixos não apenas proporciona estabilização de imagem e rastreamento preciso, mas também expande significativamente o campo de visão operacional. No contexto aeronáutico, essa integração se manifesta em sensores optoeletrônicos cruciais para operações militares e de reconhecimento. Contudo, um desafio persistente reside na manutenção do rastreamento confiável, dado o intrínseco equilíbrio entre a precisão do algoritmo de detecção e o custo computacional, bem como as dinâmicas complexas do gimbal (incluindo não linearidades e atrasos de comunicação). Este projeto de mestrado tem como objetivo principal desenvolver uma solução de controle robusta para aplicações de rastreamento de objetos em imagens em tempo real. Para tanto, emprega-se o YOLOv8, um algoritmo de detecção de última geração, cuja saída alimenta um sistema de controle PID. A atuação do controle é sobre um gimbal de dois graus de liberdade, responsável por ajustar a orientação de uma câmera para manter o objeto centralizado. A validação do desempenho das técnicas de controle propostas é realizada em um ambiente de simulação (Simulink), executado em tempo real, permitindo uma avaliação abrangente sob condições que emulam cenários operacionais.

Abstract

Real-time object detection plays a fundamental role in the advancement of autonomous and security systems. The integration of this technology with multi-axis gimbals not only provides image stabilization and precise tracking but also significantly expands the operational field of view. In the aeronautical context, this integration manifests in optoelectronic sensors crucial for military and reconnaissance operations. However, a persistent challenge lies in maintaining reliable tracking, given the intrinsic balance between detection algorithm precision and computational cost, as well as the complex dynamics of the gimbal (including nonlinearities and communication delays). This master's project aims to develop a robust control solution for real-time object tracking in acquired images. To this end, YOLOv8, a state-of-the-art object detection algorithm, is employed, whose output feeds a PID control system. The control actuation is performed on a two-degree-of-freedom gimbal, responsible for adjusting the camera's orientation to keep the object centered. The performance validation of the proposed control techniques is carried out in a simulation environment (Simulink), executed in real-time, allowing for a comprehensive evaluation under conditions that emulate operational scenarios.

List of Figures

FIGURE 2.1 – Color Image represented as a tensor. Source: Jiang <i>et al.</i> (2022). . .	26
FIGURE 2.2 – An example of NN architecture. Source: Brunton e Kutz (2022). . .	28
FIGURE 2.3 – Example of max and average pooling. Source: Yingge <i>et al.</i> (2020). . .	29
FIGURE 2.4 – Gradient descent on a simple function for different δ . Source: Murphy (2012).	30
FIGURE 2.5 – One node per layer, one hidden layer NN. Source: Brunton e Kutz (2022)	31
FIGURE 2.6 – Object localization using bounding box. Extracted from Lin <i>et al.</i> (2015).	32
FIGURE 2.7 – Precision-Recall curve interpolation. Extracted from Padilla <i>et al.</i> (2020).	34
FIGURE 2.8 – YOLO model. Source: Redmon <i>et al.</i> (2016).	35
FIGURE 2.9 – YOLOv1 architecture. Source: Redmon <i>et al.</i> (2016).	35
FIGURE 2.10 – Kalman filter algorithm working recursively. Extracted from Lacey (1998).	36
FIGURE 2.11 – Reference frames of 2-DOF Gimbal. Extracted from Rue (1969) . .	38
FIGURE 3.1 – Proposed architecture for the system.	46
FIGURE 3.2 – Image acquisition model.	47
FIGURE 3.3 – Pinhole camera geometry. Extracted from Hartley e Zisserman (2004). .	47
FIGURE 3.4 – Resultant frame.	48
FIGURE 3.5 – Pixel range for each frame.	48
FIGURE 3.6 – Kalman filter applied to the YOLOv8’s detections.	51
FIGURE 3.7 – Equivalent circuit diagram of DC servo motor. Extracted from Zhou <i>et al.</i> (2018).	53

FIGURE 3.8 – FAULHABER [®] 1524T012SR.	54
FIGURE 3.9 – Simulink model for DC servomotor and gimbals.	57
FIGURE 3.10 – Root locus plot for Elevation Axis.	58
FIGURE 3.11 – Root locus plot for Azimuth Axis.	59
FIGURE 3.12 – PID controller implemented in Elevation Axis.	61
FIGURE 4.1 – mAP@50 and mAP@50-95 evolution, respectively.	63
FIGURE 4.2 – Instances per class.	64
FIGURE 4.3 – Network communication of the control system. Extracted from Kiesel <i>et al.</i> (2022)	65
FIGURE 4.4 – Performed test for latency.	65
FIGURE 4.5 – Object recognition delay in seconds.	66
FIGURE 4.6 – Test performed for system behavior analysis.	67
FIGURE 4.7 – System response to the input signal.	67

List of Tables

TABLE 2.1 –	Types of bounding box prediction.	33
TABLE 3.1 –	Resultant equations for the Kalman filter recursive process.	50
TABLE 3.2 –	Gimbal’s Inertial Parameters.	53
TABLE 3.3 –	FAULHABER® 1524T012SR parameters	54
TABLE 3.4 –	FAULHABER® IE2-512 parameters.	55
TABLE 3.5 –	PID controller parameters for each axis.	60
TABLE 4.1 –	mAP@50 and mAP@50-95 for each class.	63

List of Abbreviations and Acronyms

TF	TensorFlow
CNN	Convolutional Neural Network
R-CNN	Region-based CNN
YOLO	You Only Look Once
SSD	Single Shot Detector
MIMO	Multi-Input, Multi-Output
DC	Direct Current
2-D	Two dimensional
NN	Neural Network
TP	True Positive
FP	False Positive
TN	True Negative
FN	False Negative
P	Precision
R	Recall
ReLU	Rectified Linear Unit
Backprop	Backpropagation Algorithm
SGD	Stochastic Gradient Descent
IoU	Intersection over Union
BB	Bounding Box
mAP	Mean Average Precision
NUC	Next Unit of Computing
PC	Personal Computer
UDP	User Datagram Protocol
2-DOF	Two-Degrees-of-Freedom
PID	Proportional-Integral-Derivative
ISR	Intelligence, Surveillance, and Reconnaissance
UAV	Unmanned Aerial Vehicle
LOS	Line-of-Sight

HIL	Hardware-in-the-Loop
SMC	Sliding Mode Control
MPC	Model Predictive Control
FLC	Fuzzy Logic Control

List of Symbols

A_k	Weight matrix
$f()$	Activation function
$x^{(k)}$	Output of the k-th layer
x	Neural network input
y	Neural network output
$S(i, j)$	Feature map
$I(m, n)$	Input image (2-D array)
$K(i - m, j - n)$	Kernel (2-D)
$\nabla f(x)$	Gradient of the function $f(x)$
$\hat{x}$	Unit vector
δ	Learning rate
E	Computed error in the neural network
y_0	Correct (expected) output of the neural network
a, b	Neural network weights (simplified example)
w_k^j	j-th component of the weight vector w at the k-th iteration
w_j	Vector of all network weights at the j-th iteration
B_p	Predicted bounding box
B_{gt}	Ground truth bounding box
$p_{interp}(r)$	Interpolated precision at recall level r
AP_i	Average Precision for class i
x_k	State vector at time k (Kalman Filter)
Φ	State transition matrix (Kalman Filter)
w_k	White noise (process noise) (Kalman Filter)
z_k	Measurement vector at time k (Kalman Filter)
H	Measurement matrix (Kalman Filter)
v_k	Measurement error (Kalman Filter)
$\hat{x}_k$	Estimated state variable (Kalman Filter)
$\hat{x}'_k$	Prior state of $\hat{x}_k$ (Kalman Filter)
K_k	Kalman Gain
I	Identity Matrix

P_k	Error covariance matrix (Kalman Filter)
Q	Covariance of w_k (Kalman Filter)
R	Covariance of v_k (Kalman Filter)
u_d^F	Unit vector of reference frame F in direction d
η	Angular position of the outer gimbal (azimuth)
ϵ	Angular position of the inner gimbal (elevation)
$\mathbb{T}_{F_1 F_2}$	Transformation matrix from reference frame F_1 to F_2
Ω_{IB}	Angular velocity vector of the inner gimbal relative to the base frame
Ω_{OB}	Angular velocity vector of the outer gimbal relative to the base frame
$\dot{\eta}$	Angular velocity of the outer gimbal (derivative of η)
$\dot{\epsilon}$	Angular velocity of the inner gimbal (derivative of ϵ)
α_{IB}	Angular acceleration vector of the inner gimbal
α_{OB}	Angular acceleration vector of the outer gimbal
$T_{friction}$	Total friction torque
$T_{viscous}$	Viscous friction torque
$T_{coulomb}$	Coulomb friction torque
k_{vf}	Viscous friction coefficient
Ω	Angular velocity (general for friction)
k_{cf}	Coulomb friction coefficient
M_{OI}	Moment applied from the outer gimbal to the inner gimbal
$H_{I,P}^I$	Angular momentum of the inner gimbal about pivot point P
$\mathbb{I}_P^I$	Inertia tensor of the decoupled inner gimbal about pivot point P
I_x^I, I_y^I, I_z^I	Moments of inertia of the inner gimbal about x, y, z axes
$\alpha_{IB_x}, \alpha_{IB_y}, \alpha_{IB_z}$	Components of the inner gimbal's angular acceleration along x, y, z axes
$\Omega_{IB_x}, \Omega_{IB_y}, \Omega_{IB_z}$	Components of the inner gimbal's angular velocity along x, y, z axes
T_{m_I}	Motor torque transmitted to the inner gimbal
$T_{Friction_{OI}}$	Friction torque between the outer and inner gimbals
$H_{O,P}^O$	Angular momentum of the outer gimbal about pivot point P
$\mathbb{I}_P^O$	Inertia tensor of the decoupled outer gimbal about pivot point P
M_{BO}	Moment applied from the gimbal base to the outer gimbal
M_{IO}	Moment applied from the inner gimbal to the outer gimbal
$T_{Friction_{BO}}$	Friction torque between the base and the outer gimbal
T_{m_O}	Motor torque applied to the outer gimbal
I_x^O, I_y^O, I_z^O	Moments of inertia of the outer gimbal about x, y, z axes
$u(t)$	Applied motor voltage
R_a	Armature resistance of the motor
L_a	Armature inductance of the motor
i_a	Armature current of the motor
e_a	Counter electromotive force (back-EMF)

ω	Rotational speed (of the motor)
M_c	Load torque (of the motor)
K_e	Counter electromotive force (back-EMF) constant
$\dot{\theta}(t)$	Angular velocity (derivative of angular position)
C_m	Motor torque constant
$\Theta(s)$	Laplace transform of angular position
$T_{m_I}(s)$	Laplace transform of motor torque (inner gimbal)
$u(s)$	Laplace transform of input voltage
s	Laplace operator
$C(s)$	PID controller transfer function
K_p	Proportional gain of PID
K_i	Integral gain of PID
K_d	Derivative gain of PID
$G(s)$	Plant transfer function
u, v	Pixel coordinates (2D)
x_n, y_n	Normalized image coordinates
K	Camera intrinsic matrix
f_x, f_y	Focal length in x and y directions
c_x, c_y	Principal point coordinates
x_c, y_c, z_c	Coordinates in the camera reference frame

Contents

1	INTRODUCTION	20
1.1	Objective	20
1.2	Motivation	20
1.3	Contribution of the Work	22
1.4	Literature Review	22
1.5	Organization	25
2	THEORETICAL BACKGROUND	26
2.1	Digital Images	26
2.2	Machine Learning	27
2.3	Neural Networks	27
2.4	Convolutional Neural Networks	28
2.4.1	Convolution	28
2.4.2	Pooling	29
2.5	Gradient Descent	30
2.6	Backpropagation Algorithm	30
2.7	Stochastic Gradient Descent Algorithm	31
2.8	Object Detection	32
2.8.1	IoU	32
2.8.2	Mean Average Precision	33
2.9	YOLO - You Only Look Once	34
2.10	Kalman Filter	35
2.11	Gimbal System	37

2.11.1	Reference Frames	37
2.11.2	Kinematics	39
2.11.3	Friction Model	40
2.11.4	Gimbal Dynamics	41
3	OBJECT TRACKING SYSTEM	45
3.1	Architecture	45
3.2	Image Acquisition	46
3.3	Object Detection Algorithm	48
3.4	State Estimation	49
3.4.1	Post Filter Processing	51
3.5	Gimbal Model	52
3.5.1	Rotational Axes	52
3.5.2	Electrical Motors	53
3.5.3	Transfer Functions	55
3.5.4	PID Controller	57
4	RESULTS AND DISCUSSIONS	62
4.1	Training YOLO	62
4.1.1	Dataset	62
4.1.2	Evaluation of Results	62
4.2	System Performance	64
4.2.1	Latency	64
4.2.2	Controlled Output	66
5	CONCLUSIONS AND FUTURE WORKS	68
5.1	Conclusions	68
5.2	Future Works	69
	BIBLIOGRAPHY	71

1 Introduction

This work presents the modeling of a system capable of detect and track objects for several applications in Surveillance and Monitoring. Objective, motivation, literature review, and the organization of this work are presented in the following sections.

1.1 Objective

In the context of real-time visual object tracking systems, a persistent challenge lies in achieving reliable performance, especially when considering the complex dynamics of the actuator and the communication system. The accuracy of tracking in images, acquired and processed in real-time, critically depends on the ability to mitigate inherent nonlinearities in mechatronic systems and to compensate for communication delays, ensuring that the object of interest remains centered within the field of view. In this scenario, this project aims to develop a comprehensive control solution for object tracking applications in images.

1.2 Motivation

In recent years, technology has transformed many sectors, which are now significantly benefiting from systems capable of detecting and tracking objects. These systems prove invaluable in diverse situations, ranging from enhancing security applications to optimizing industrial production, enabling the advancement of autonomous vehicles and empowering robots to perform complex tasks. The ability to perceive and track objects in real-time not only boosts efficiency but also considerably improves the safety and accuracy of numerous daily activities and critical operations. In public safety, for instance, cameras equipped with these technologies are indispensable for monitoring busy areas, deterring criminal activities, and assisting in investigations. They can identify suspicious behavior and instantly alert law enforcement, providing detailed imagery as evidence. This marks a substantial progression from traditional surveillance methods, which are heavily reliant on human operators. Within the automotive sector, autonomous vehicles depend

on these systems to navigate securely, accurately identifying pedestrians, other vehicles, and obstacles on the road. Furthermore, these systems empower vehicles to make swift and precise decisions, adapting seamlessly to environmental changes. As we move closer to a future characterized by increased vehicle autonomy, the imperative to continuously enhance these systems becomes ever more evident. In military operations, object detection and tracking are paramount for surveillance and security. Drones outfitted with these technologies monitor conflict-affected zones, offering a detailed, real-time perspective of the theater of operations. These systems are vital for identifying and tracking threats, such as adversarial vehicles and equipment, thereby augmenting operational effectiveness and mitigating risks for personnel. The speed and precision in target detection are critical determinants of success in military missions, and advancements in this domain can profoundly impact critical situations.

Integrating object detection and tracking technologies with multi-axis gimbal systems elevates the performance of these applications to a new level. Gimbals provide a mechanical platform capable of isolating a sensor from undesirable base movements (image stabilization) and actively reorienting the Line-of-Sight (LOS) to follow a target (precise tracking). This integration not only yields more stable and smoother footage, crucial for the quality of visual information, but also drastically extends the sensor's operational field of view, enabling the monitoring of distant or fast-moving targets across expansive areas.

In the aeronautical sector, the application of gimbals equipped with optoelectronic sensors is particularly pertinent. Such solutions form the core of advanced Intelligence, Surveillance, and Reconnaissance (ISR) systems utilized in Unmanned Aerial Vehicles (UAVs). In these operations, the ability to maintain a target of interest at the center of the camera's field of view, regardless of the aircraft's or the target's own movements, is crucial for collecting high-quality data and ensuring mission effectiveness.

Despite significant progress in the field, developing robust and high-performance visual tracking systems still presents considerable challenges. The inherent complexity of gimbal dynamics, which involves nonlinearities (such as friction) in addition to communication delays intrinsic to distributed system architectures, poses an obstacle to achieving precise and reliable tracking. The balance between object detection accuracy and the computational cost required for real-time processing adds another layer of complexity to this challenge.

Within this context, this Master's thesis aims to develop a control solution for gimbal-based object tracking systems using real-time acquired images. The motivation stems from the existing gap between simplified theoretical models and the actual behavior of complex mechatronic systems. By modeling the dynamics and nonlinearities of the gimbal and its actuators, combined with the design of a controller capable of mitigating the effects of disturbances and delays, this work seeks to contribute to improving the performance

of visual tracking systems in engineering applications.

1.3 Contribution of the Work

While adopting strategic simplifications in its modeling to maintain clarity and project feasibility, this work offers relevant academic and practical contributions in the field of visual object tracking through gimbal systems.

The study proposes an integrated and complete system architecture, uniting visual perception (YOLOv8 algorithm), data communication (UDP protocol), and motion control (PID controller) with the electromechanical dynamics of a two-degrees-of-freedom gimbal. This end-to-end integration, simulated in a unified environment, represents a fundamental step for autonomous tracking systems. The system modeling, despite the inherent simplicity of the linearized model for controller design, explicitly and modularly incorporates the most critical nonlinearities for gimbal behavior, such as Coulomb friction, as well as the inclusion of a communication delay via Padé approximation, which adds an important level of realism. Validation is performed in a Simulink environment, with a practical focus on real-time, allowing for a more faithful evaluation of controller performance. Additionally, the developed model establishes a clear platform for future research and for the introduction of additional complexities, by isolating and modeling primary nonlinearities as disturbances, offering a didactic starting point for the development of more advanced control techniques or the incorporation of further details.

1.4 Literature Review

A methodology for detecting and tracking objects in images is proposed by Mane e Mangale (2018). The approach consists of using a CNN (convolutional neural network) to perform two procedures, object detection and tracking. In this work, the TF (TensorFlow) tool is presented to simplify the construction of the object detection algorithm.

Girshick *et al.* (2014) present a system that perform object detection in three steps. The first generates category-independent region proposal, and these proposals define the set of candidate detections available. The second module is a large convolutional neural network that extracts a fixed-length feature vector from each region. The third module is a set of class specific linear SVMs. This system is called R-CNN (*Region-based Convolutional Neural Networks*).

R-CNN achieves excellent object detection accuracy according to Girshick (2015), however, it has some drawbacks, such as expensive training and slow object detection.

By making changes on the training and test methods, Girshick (2015) presented a new training algorithm, called Fast R-CNN. As the name suggests, Fast R-CNN is faster to train and test in relation to R-CNN.

Ren *et al.* (2017) presents a different region proposal method, called Faster R-CNN. Faster R-CNN is composed of two modules, where the first is a deep fully convolutional network that proposes regions, and the second is a Fast R-CNN that uses the proposed regions. This new approach can achieve an speed of 5-17 *frames per second*, which is significantly higher than other region proposal networks.

Redmon *et al.* (2016) proposes a solution for detecting objects in videos, called “You Only Look Once” (YOLO). This methodology differs because it unifies different components of object detection in a single neural network, thus managing to use information from the image as a whole to predict each bounding box (region of the image that locates the detected object). The YOLO model is implemented using a CNN.

An application of YOLO in its most current version (YOLOv8) to solve a surveillance problem, by detecting and tracking movement in real time, is presented in Al-E’mari *et al.* (2024). The approach was to use a dataset for security applications and augment the YOLOv8 architecture with security-specific algorithms to improve real-time threat detection.

SSD (Single Shot Detector) algorithm is proposed by Liu *et al.* (2016). According to Liu *et al.* (2016), algorithms such as Faster R-CNN are accurate, however, for embedded systems is computationally intensive, and the cost of increasing speed in these approaches comes with significantly decrease in detection accuracy. The results on the Pascal VOC2007 test shows that SSD can achieve similar accuracy to Faster R-CNN, with higher speed.

The work done by Vinaykumar e Jatoth (2014) presents a comparison between two algorithms for solving the problem of tracking objects in images, the Kalman filter and the alpha-beta filter. Using a sphere as the object of study, performance is given by analyzing parameters such as the estimate of the centroid, the radius of the sphere estimated by the algorithm and the iteration time. The comparison between the algorithms shows greater accuracy in the results presented by the use of the Kalman filter.

In Cunha *et al.* (2019), an airborne inertial platform (“*Gimbal*”) is controlled based on images. In order to achieve real-time performance, the author points to the use of algorithms such as YOLO and SSD, which can be called “one-stage detectors” and can speed up the process of locating objects in an image.

Rahmat *et al.* (2016) conducted a comprehensive study on a two-degrees-of-freedom (2-DOF) gun turret model, providing a detailed derivation of its dynamics and simulating its behavior in MATLAB-Simulink. Their work specifically evaluated and compared the performance of PID and Sliding Mode Control (SMC) strategies for accurate target angle

tracking, demonstrating the effectiveness of the SMC controller in generating sufficient drive torque and precisely reducing system error.

Galal *et al.* (2009) propose a solution for the control of a gun-turret system, a complex multi-body, multi-input, multi-output (MIMO) nonlinear system, utilizing Fuzzy Logic Control (FLC). This methodology addresses the challenges of achieving rapid and precise tracking responses in weapon systems, particularly under the influence of disturbances, nonlinearities, and modeling uncertainties that often degrade the performance of conventional controllers. The work details the application of a fuzzy scheme for compensation and nonlinear feedback control laws, which are crucial for managing the coupled dynamics of the turret and barrel driven by two DC motors. The authors highlight that FLC provides an effective means of capturing the approximate, inexact nature of the real world and adapting to unexpected parameter variations, thereby offering a robust and adaptive control strategy that is designed to accommodate inherent system uncertainties. The effectiveness of their fuzzy logic controller design is validated through simulation results, demonstrating improved system tracking performance.

Tong *et al.* (2020) investigated the gimbal torque and coupling torque in a six-degrees-of-freedom (6-DOF) magnetically suspended yaw gimbal, exploring various control methods such as PID, fuzzy control, Sliding Mode Control (SMC), adaptive control, feed-forward control, disturbance observers, and H-infinity control, and specifically employing decentralized PD control for radial active magnetic bearings. Their research focused on developing dynamic models of the yaw gimbal and a three-axis inertially stabilized platform (ISP), analyzing coupling torques among the gimbals, and studying the characteristics of gimbal torque generated by the magnetic suspension system. The experimental results indicated that the magnetic suspension system's gimbal torque can effectively compensate for coupling torques, thereby improving the yaw gimbal's attitude stabilization precision.

Poyrazoğlu (2017) presented a detailed modeling and control strategy for a two-degrees-of-freedom (2-DOF) gimbal system, employing the Newton-Euler approach to incorporate significant nonlinearities such as friction and mass unbalance. The study further explored state-space representations for both coupled and decoupled gimbal models, designing and comparing cascade PI and Linear Quadratic Integral (LQI) controllers for stabilization and target tracking within a MATLAB/Simulink virtual environment.

Ramírez (2016) presented a work on the design and implementation of a control system for a two-degrees-of-freedom (2-DOF) gimbal, intended for UAV intelligence, surveillance, and reconnaissance missions. His work focused on developing a control structure to stabilize the camera's optical axis against environmental disturbances, deriving equations of motion using the Newton-Euler method, and validating Sliding Mode Control (SMC) algorithms through both simulation and experimental tests on a physical prototype, comparing their performance against a Proportional-Integral (PI) compensator.

1.5 Organization

This work is organized into 7 chapters, following a logical progression that spans from theoretical fundamentals and detailed system modeling to controller design, its validation through simulation, and finally, conclusions and suggestions for future work.

Chapter 1 establishes the general context of the research followed by the motivation, definition of the problem to be investigated, and the objectives and expected contributions of this study, whilst presenting an overview of existing literature.

Chapter 2 addresses the fundamental concepts of computer vision, neural networks, gimbals and stabilized platforms, vision-based object tracking, and control of pointing systems, in addition to analyzing related works that support the development of this project.

Chapter 3 is dedicated to the construction of the complete system's mathematical model, detailing the gimbal's kinematics and dynamics, the DC motor's modeling with its essential nonlinearities, the sensors used for feedback, and the representation of communication delay.

Chapter 4 describes the methodology used to design and tune the controller, specifying the chosen control strategy, the process of preparing the visual tracking input signal, and the performance analysis of the controller.

Chapter 5 presents the simulation environment and the obtained results, validating the proposed control system's performance.

Finally, Chapter 6: Results and Discussion (or Chapter 7: Conclusions and Future Work, depending on the final numbering) consolidates the main conclusions of the work, discusses its limitations, and proposes directions for future research in the field.

2 Theoretical Background

This section will provide an overview on computer vision, object detection and tracking on digital images, neural networks, Kalman filter, and modeling and control of a gimbal system.

2.1 Digital Images

To begin an explanation on computer vision, we need to understand how digital images are made. Gonzalez e Woods (2006) shows that digital images are obtained through sensors arranged in the form of a 2-D (two dimensional) array, and the response of each sensor is proportional to the integral of the light energy projected onto the surface of the sensor. When the sensors captures the image, two processes are made to digitize, Sampling is used to digitize the coordinates and Quantization for intensity (amplitude). The resulting representation of digital images in grayscale can be approximated by a traditional matrix, where each element represents a pixel. Color images can be represented as a tensor of order three. “A tensor is a multidimensional array... a first-order tensor is a vector, a second-order tensor is a matrix, and tensors of order three or higher are called higher-order tensors” (KOLDA; BADER, 2009). Figure 2.1 illustrates color images as a tensor in the RGB (Red, Blue, Green) system.

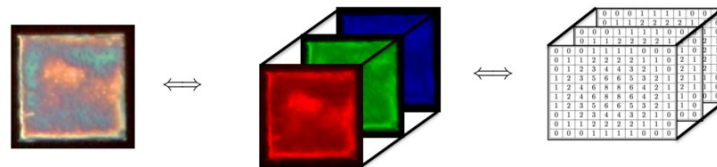


FIGURE 2.1 – Color Image represented as a tensor. Source: Jiang *et al.* (2022).

It is important to have this consideration when thinking about the input datasets.

2.2 Machine Learning

Murphy (2012) defines machine learning as a set of methods that can automatically detect patterns in data, and then use the uncovered patterns to predict future data, or to perform other kinds of decision making under uncertainty (such as planning how to collect more data). In this field, Learning Algorithms can be defined as proposed by Mitchell (1997): “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ”. Machine learning can be divided as Supervised machine learning and Unsupervised machine learning. In the Supervised learning the algorithm is presented with labeled datasets, explicitly defining the desired input-output behavior of the model. On the other hand, in Unsupervised learning no labels are given, so the patterns must be found in data in order to determine how to cluster the information and generate/create labels for predicting and classifying new data. For more information, refer to Brunton e Kutz (2022).

2.3 Neural Networks

As stated in Brunton e Kutz (2022), the Nobel Prize winning work of Hubel e Wiesel (1962) on the primary visual cortex of cats served as inspiration for NNs (neural networks). NNs specifically optimize over the compositional function

$$\underset{\mathbf{A}_j}{\operatorname{argmin}}(f_M(\mathbf{A}_M, \dots, f_2(\mathbf{A}_2, f_1(\mathbf{A}_1, \mathbf{x}))\dots) + \lambda g(\mathbf{A}_j)), \quad (2.1)$$

where each matrix $\mathbf{A}_k$ refers to the weights connecting the NN from the k -th to the $(k+1)$ -th layer. Being a massively under-determined system, $\lambda g(\mathbf{A}_j)$ (called penalization) tries to regulate it. Some of the algorithms used to solve this equation are backpropagation (section 2.6) and stochastic gradient descent (section 2.7). The number of layers, dimension, connections and other design features of NNs are being studied and may vary according the application. Figure 2.2 shows the architecture of a generic NN mapping $\mathbf{x}$ to $\mathbf{y}$.

If we consider a linear mapping between each layer, it becomes a linear system that can be represented as $\mathbf{A}_k \mathbf{x}^{(k-1)} = \mathbf{x}^{(k)}$. When working with nonlinear transformation from input to output, the mapping in each layer can be represented as

$$\mathbf{x}^{(k)} = f(\mathbf{A}_k, \mathbf{x}^{(k-1)}), \quad (2.2)$$

where f is called an activation function, and works as a transfer function. Linear, sigmoid,

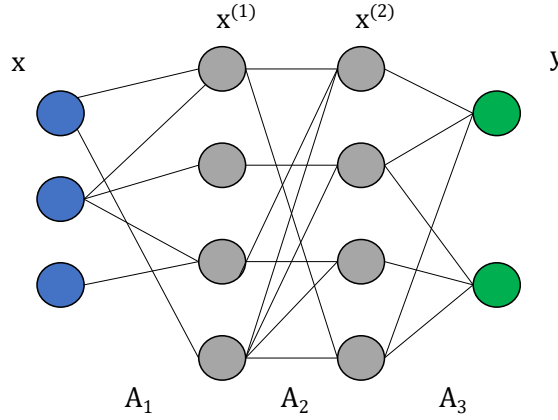


FIGURE 2.2 – An example of NN architecture. Source: Brunton e Kutz (2022).

$\tanh$, and rectified linear unit (ReLU) are some of the commonly used activation functions, respectively expressed as

$$f(x) = x, \quad (2.3)$$

$$f(x) = \frac{1}{1 + \exp(-x)}, \quad (2.4)$$

$$f(x) = \tanh(x), \quad (2.5)$$

$$f(x) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases} \quad (2.6)$$

2.4 Convolutional Neural Networks

Goodfellow *et al.* (2016) give a brief description on how CNNs (Convolutional Neural Networks) work: a typical layer of a convolutional network consists of three stages. In the first stage, the layer performs several convolutions in parallel to produce a set of linear activations. In the second stage, each linear activation is run through a nonlinear activation function, such as the ReLU activation function. This stage is sometimes called the detector stage. In the third stage, we use a pooling function to further modify the layer's output.

2.4.1 Convolution

The convolution of two functions $f(t)$ and $g(t)$ can be written as

$$s(t) = (f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau. \quad (2.7)$$

When working with CNNs, the first argument is called input, the second argument

is referred as kernel and the output is the feature map. If we consider an image I (2-D array) as input, it is preferred to use a kernel K that is also 2-D. In this case, the feature map can be expressed as

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n). \quad (2.8)$$

Through the commutative property of convolution, the cross-correlation function is usually found on neural networks libraries as

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n). \quad (2.9)$$

Another interesting aspect of convolution is the possibility to work with different sized inputs. Computational effort is reduced in CNNs by making the kernel smaller than the input, and it is called sparse connectivity. This allows the network to detect small meaningful features in the input, such as edges in images.

2.4.2 Pooling

A pooling function replaces the output of the network at a certain location with a summary statistic of the nearby outputs (GOODFELLOW *et al.*, 2016). The pooling operation is used to avoid small variances on the input. This is an effective strategy to help control overfitting and fit the computation in memory. A common example is the max pooling. For example, consider a kernel K as a 2-D array on an input image I . At every iteration, the convolution between the kernel and the image $(I * K)$ results in 4 values. Max pooling applies the max function on the resulting values. Figure 2.3 illustrates two pooling operations: max pooling and average pooling.

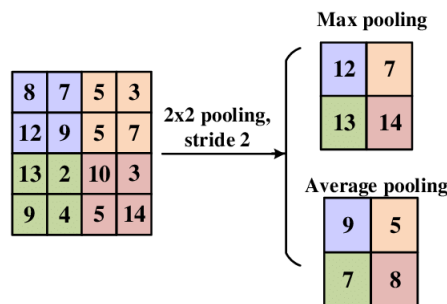


FIGURE 2.3 – Example of max and average pooling. Source: Yingge *et al.* (2020).

2.5 Gradient Descent

The extremum of a multi-dimensional function $f(x)$, either maximum or minimum, will have a gradient equivalent to zero, so

$$\nabla f(x) = 0. \quad (2.10)$$

When working with high dimensional systems, the presence of saddles requires testing if the extremum point is a maximum or minimum. The idea of gradient descent is to iterate over the information obtained by differentiating $f(x)$ until it gets to a local minimum. To simplify the illustration,

$$f(x) = x^2. \quad (2.11)$$

Thus, the gradient can be defined as

$$\nabla f(x) = \frac{\partial f(x)}{\partial x} \hat{\mathbf{x}}, \quad (2.12)$$

where $\hat{\mathbf{x}}$ is a unit vector. In order to find the local minimum, the idea is to follow the direction given by $-\nabla f(x)$. So, the algorithm can be generalized as

$$\mathbf{x}_{k+1}(\delta) = \mathbf{x}_k - \delta \nabla f(x_k), \quad (2.13)$$

where δ is the parameter that dictates how far each “step” goes. Figure 2.4 illustrates the Gradient descent finding a local minimum for different values of δ .

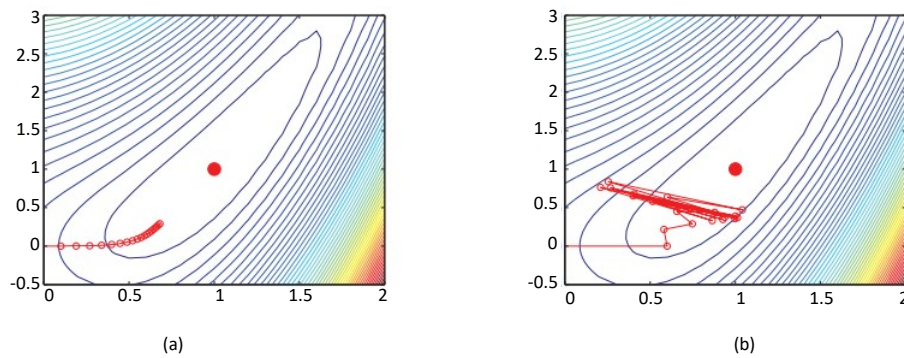


FIGURE 2.4 – Gradient descent on a simple function for different δ . Source: Murphy (2012).

2.6 Backpropagation Algorithm

The backpropagation algorithm (backprop) exploits the compositional nature of NNs in order to frame an optimization problem for determining the weights of the network

(BRUNTON; KUTZ, 2022). In simple words, backprop is a efficient gradient estimation method that can serve as input for gradient descent. As an example, consider a NN which has one node per layer, and only one hidden layer, as shown in Figure 2.5.

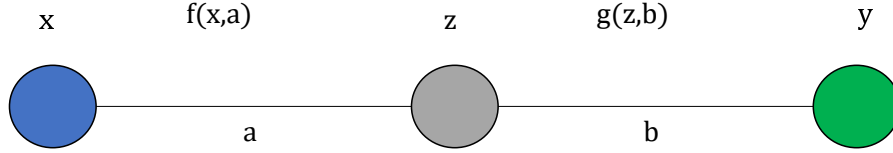


FIGURE 2.5 – One node per layer, one hidden layer NN. Source: Brunton e Kutz (2022)

The relation between the input and output in this NN can be described as

$$y = g(z, b) = g(f(x, a), b), \quad (2.14)$$

where f and g are activation functions and a and b the weights. Thus, the output error E can be computed as

$$E = \frac{1}{2}(y_0 - y)^2, \quad (2.15)$$

where y_0 is the correct output (imagine a labeled dataset) and y is the output given by the NN. Backprop works on finding a and b that minimizes E . In this simple case, the weights are updated iteratively

$$a_{k+1} = a_k - \delta \frac{\partial E}{\partial a_k}, \quad (2.16)$$

$$b_{k+1} = b_k - \delta \frac{\partial E}{\partial b_k}, \quad (2.17)$$

where δ is a hyperparameter called learning rate. A full generalization of backprop can be written as

$$w_{k+1}^j = w_k^j - \delta \frac{\partial E}{\partial w_k^j}, \quad (2.18)$$

where w_k^j can be understood as the j -th component of a vector $\mathbf{w}$ that contains all the elements of the matrices $\mathbf{A}_j$ (shown in Section 2.3).

2.7 Stochastic Gradient Descent Algorithm

As described by Brunton e Kutz (2022), the Stochastic Gradient Descent Algorithm (SGD) has an advantage when it comes to training a NN: faster evaluation of the optimal weights. SGD estimates the gradient using a batch of a fixed number of training examples, which is less than the actual dataset (mini-batch) at each iteration. Thus, the SGD

algorithm can be written as

$$\mathbf{w}_{j+1}(\delta) = \mathbf{w}_j - \delta \nabla E_k(\mathbf{w}_j), \quad (2.19)$$

where $\mathbf{w}_j$ is the vector of all the network weights at the j -th iteration, and $k \in [k_1, k_2, \dots, k_p]$ denotes the p selected data points k_j used to approximate the gradient.

2.8 Object Detection

Lin *et al.* (2015) defines image classification as a task that requires binary labels, in order to indicate if objects are present in an image. Detecting an object involves two key tasks: confirming the presence of an object from a specified class and identifying its location within the image. The location is usually represented by a bounding box, as shown in Figure 2.6.

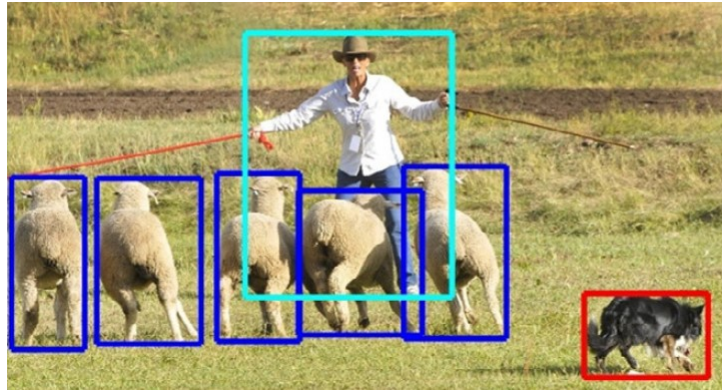


FIGURE 2.6 – Object localization using bounding box. Extracted from Lin *et al.* (2015).

2.8.1 IoU

Intersection over Union (IoU) measures the closeness of the predicted and ground truth BBs. As the name suggests, it is the area of intersection between the predicted BB and the ground truth BB divided by the area of their union. This parameter dictates how close the predicted BB is from the ground truth and can be written as

$$IoU = \frac{\text{area}(B_p \cap B_{gt})}{\text{area}(B_p \cup B_{gt})} \quad (2.20)$$

where B_p and B_{gt} are the predicted and ground truth bounding boxes, respectively. A prediction is considered correct if the predicted and ground truth BBs belongs to the same class and the IoU between them is higher than a threshold value. Predictions can be separated into four categories, as shown in Table 2.1.

TABLE 2.1 – Types of bounding box prediction.

Prediction	Description
True Positive (TP)	Model predicts a bounding box on a location and it is correct
False Positive (FP)	Model predicts a bounding box on a location and it is wrong
True Negative (TN)	Model does not predict a bounding box on a location and it is correct
False Negative (FN)	Model does not predict a bounding box on a location and it is wrong

2.8.2 Mean Average Precision

To understand the concept of mean Average Precision (mAP), precision and recall must be defined. According to Everingham *et al.* (2010), recall is defined as the proportion of all positive examples with confidence above a given threshold. Precision is the proportion of all examples above that threshold which are from the positive class. In mathematical terms,

$$P = \frac{TP}{TP + FP}, \quad (2.21)$$

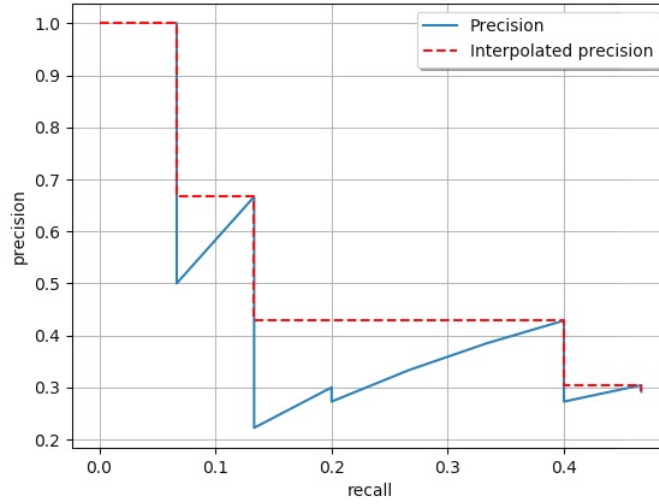
$$R = \frac{TP}{TP + FN}, \quad (2.22)$$

where P and R are precision and recall, respectively. In multiclass classification, the model outputs the conditional probability that a specific bounding box belongs to one possible class. Greater probability means higher chances for the bounding box to contain that class. The probability distribution along with a user-defined confidence threshold (between 0 and 1) value is used to classify a bounding box (GRANDINI *et al.*, 2020). A precision-recall curve plots the value of precision against recall for different confidence threshold values. To compute the Average Precision for each class, the methodology used in Pascal VOC2008 (EVERINGHAM *et al.*, 2010) divided the recall axis into eleven points $r \in [0, 0.1, \dots, 1]$ and interpolated the curve using

$$p_{\text{interp}}(r) = \max_{\tilde{r}: \tilde{r} \geq r} p(\tilde{r}), \quad (2.23)$$

which means to take the precision at each recall level r and replace by the maximum precision measured in $\tilde{r} > r$. Figure 2.7 presents an example of this interpolation on a Precision-Recall curve.

The average precision AP was then computed by taking the arithmetic mean of the

FIGURE 2.7 – Precision-Recall curve interpolation. Extracted from Padilla *et al.* (2020).

eleven precision values $p_{\text{interp}}(r)$:

$$AP = \frac{1}{11} \sum_{r \in \{0, 0.1, \dots, 1\}} p_{\text{interp}}(r). \quad (2.24)$$

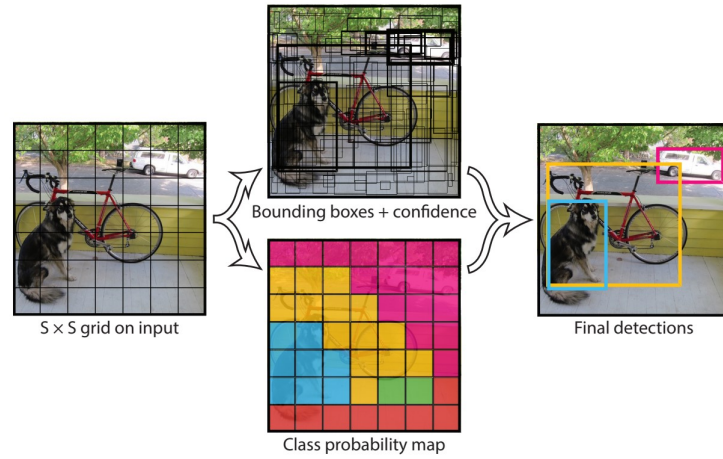
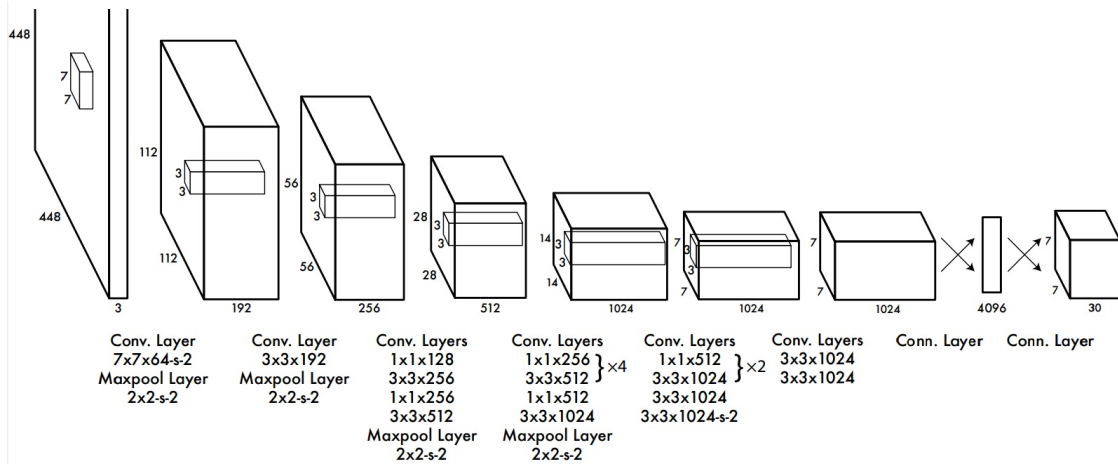
The mean Average Precision at a given IoU threshold t ($\text{mAP}@t$) can be obtained using

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i, \quad (2.25)$$

where AP_i is the Average Precision for class i . The threshold t can also be a range of numbers ($t = [50, 55, 60, \dots, 95]$). In this case, $\text{mAP}@50-95$ is the mean of mAPs computed for each value in t .

2.9 YOLO - You Only Look Once

YOLO is a computer vision model for object detection, classification and segmentation tasks in real time. The first version was presented by Redmon *et al.* (2016), and consists of a single CNN predicting multiple bounding boxes and class probabilities for those boxes simultaneously, as shown in Figure 2.8. YOLO's first version consisted of 24 convolutional layers followed by 2 fully connected layers, as shown in Figure 2.9.

FIGURE 2.8 – YOLO model. Source: Redmon *et al.* (2016).FIGURE 2.9 – YOLOv1 architecture. Source: Redmon *et al.* (2016).

In this work we will use YOLOv8, which is described by Jocher *et al.* (2023) as a model that builds upon the success of previous YOLO versions and introduces new features and improvements to further boost performance and flexibility.

2.10 Kalman Filter

According to Grewal and Andrews (2015), the Kalman filter is a mathematical tool used almost exclusively for two purposes: estimation and performance analysis of estimators. It works on minimizing the mean squared error between the actual and estimated data, providing the best estimation of the data in this sense.

To explain the algorithm, we assume that we want to determine the value of a variable in the form:

$$x_{k+1} = \Phi x_k + w_k, \quad (2.26)$$

where x_k is the state vector at time k , Φ is the state transitioning matrix from state x_k to x_{k+1} and w_k is called white noise. The idea is to consider measurements on this variable modelled as

$$z_k = \mathbf{H}x_k + v_k, \quad (2.27)$$

where z_k is the measurement of x at time k , $\mathbf{H}$ is the connection between the state vector and the measurement vector, and v_k is the associated measurement error. Now, if we consider the estimated variable $\hat{x}_k$ to be related with the measurement z_k by

$$\hat{x}_k = \hat{x}'_k + K_k(z_k - \mathbf{H}\hat{x}'_k), \quad (2.28)$$

where $\hat{x}'_k$ is the prior state of $\hat{x}_k$ and K_k is the Kalman gain, the algorithm recursively update the variables according Figure 2.10, where I is the identity matrix, P_k is the error covariance matrix and Q is the coavriance of w_k , given by

$$P_k = E[(x_k - \hat{x}_k)(x_k - \hat{x}_k)^T], \quad (2.29)$$

and

$$Q = E[w_k w_k^T]. \quad (2.30)$$

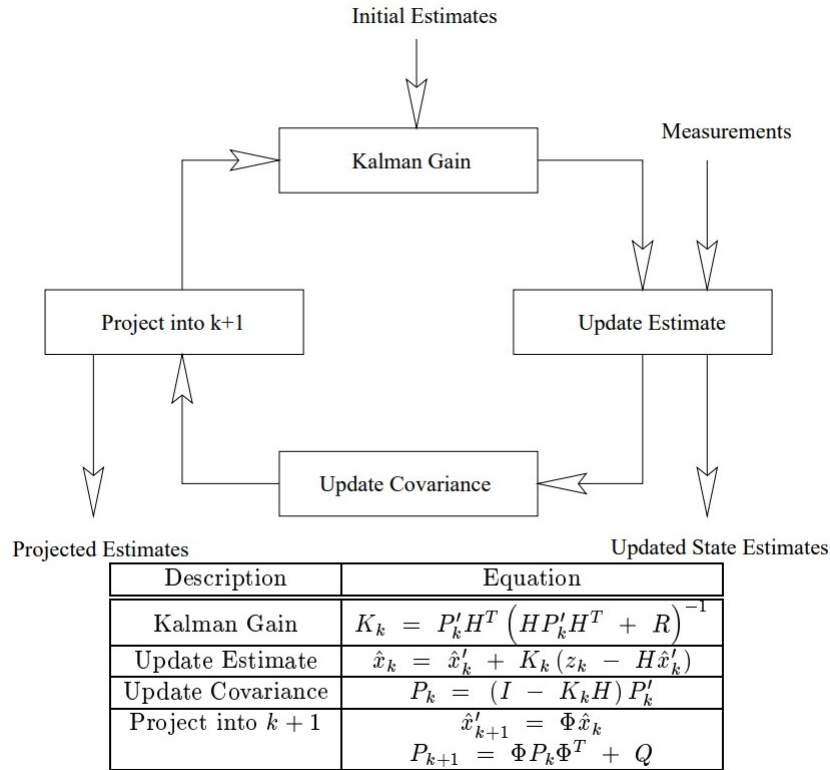


FIGURE 2.10 – Kalman filter algorithm working recursively. Extracted from Lacey (1998).

2.11 Gimbal System

This section describes the gimbal as a system, presenting the principles and how to mathematically model the dynamics.

2.11.1 Reference Frames

To understand and accurately model a 2-DOF gimbal, it is essential to first divide the system into its primary components: the inner gimbal, the outer gimbal, and the base. The inner and outer gimbals provide movement in elevation (ϵ) and azimuth (η), respectively, on their own reference frame. The base serves as the reference frame upon which the gimbals operate, and in this work, it is considered the inertial frame, meaning it is static and unaffected by external disturbances. It is assumed that all reference frames are orthogonal and their origin overlaps. Figure 2.11 illustrates the reference frames in a 2-DOF gimbal, where $\mathbf{u}_d^F$ represents the unit vector of reference frame “ F ” in direction “ d ”, in which:

B is the reference frame fixed to the gimbal base. In our case, coincident to inertial frame (earth).

O is the reference frame fixed to the outer gimbal.

I is the reference frame fixed to the inner gimbal.

And the unit vectors are expressed as:

$$\mathbf{u}_x = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{u}_y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{u}_z = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

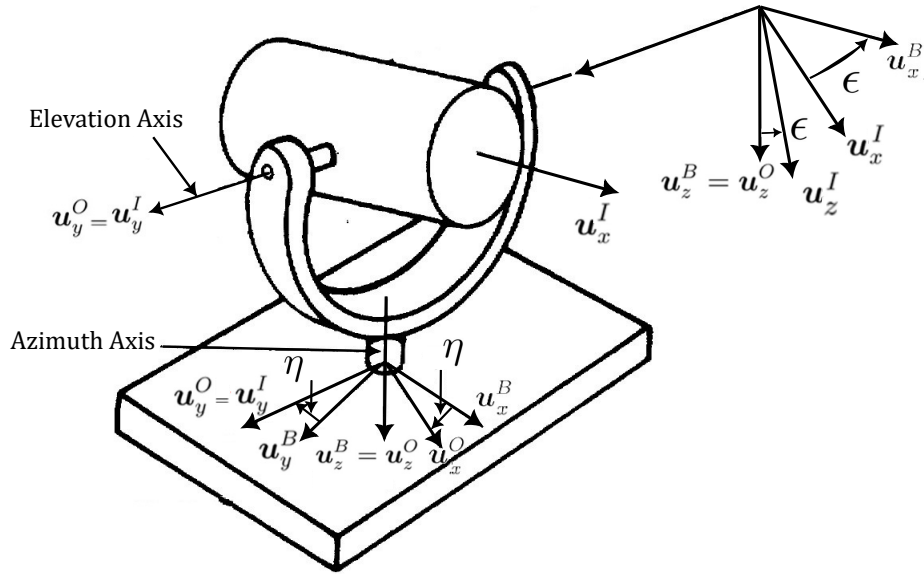


FIGURE 2.11 – Reference frames of 2-DOF Gimbal. Extracted from Rue (1969)

The relation between reference frames is established by transformation matrices, which are thoroughly discussed in Done (1999). For instance, a transformation matrix $\mathbb{T}_{F_1 F_2}$ projects a vector from reference frame F_1 to reference frame F_2 . Taking into account that our system is not affected by disturbances in the base B , the transformation matrices applied are:

$$\mathbb{T}_{BO}(\eta) = \mathbb{T}_{OB}^T(\eta) = \begin{bmatrix} \cos(\eta) & -\sin(\eta) & 0 \\ \sin(\eta) & \cos(\eta) & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad (2.31)$$

$$\mathbb{T}_{OI}(\epsilon) = \mathbb{T}_{IO}^T(\epsilon) = \begin{bmatrix} \cos(\epsilon) & 0 & \sin(\epsilon) \\ 0 & 1 & 0 \\ -\sin(\epsilon) & 0 & \cos(\epsilon) \end{bmatrix}. \quad (2.32)$$

Given the orthogonal nature of reference frames, a fundamental characteristic of transformation matrices is observed: the multiplication of a transformation matrix by its transpose invariably produces the identity matrix ($\mathbf{I}$):

$$\mathbb{T}^T \mathbb{T} = \mathbf{I}. \quad (2.33)$$

2.11.2 Kinematics

The Kinematics of the gimbal refer to the angular velocity and acceleration terms, which can be represented by 12 equations. To represent the inner and outer gimbal kinematics related to the inertial frame, the frames must be transformed to the reference frame of the gimbal base. A more detailed explanation is given in Poyrazoğlu (2017).

2.11.2.1 Angular Velocity

To express the velocity of each gimbal, firstly we establish the reference, which in our case will be the base (or inertial) frame. Hence, the velocity vector of inner and outer gimbals will be expressed as $\boldsymbol{\Omega}_{IB} = [\Omega_{IB_x}, \Omega_{IB_y}, \Omega_{IB_z}]^T$ and $\boldsymbol{\Omega}_{OB} = [\Omega_{OB_x}, \Omega_{OB_y}, \Omega_{OB_z}]^T$, respectively. Therefore, the velocity vectors can be written as

$$\Omega_{OB_x} = 0, \quad (2.34)$$

$$\Omega_{OB_y} = 0, \quad (2.35)$$

$$\Omega_{OB_z} = \dot{\eta}, \quad (2.36)$$

for the outer gimbal, and

$$\boldsymbol{\Omega}_{IB} = \dot{\epsilon} \mathbf{u}_y + \mathbb{T}_{IO}(\epsilon) \boldsymbol{\Omega}_{OB} = \dot{\epsilon} \mathbf{u}_y + \mathbb{T}_{IO}(\epsilon) \dot{\eta} \mathbf{u}_z, \quad (2.37)$$

$$\Omega_{IB_x} = -\dot{\eta} \sin(\epsilon), \quad (2.38)$$

$$\Omega_{IB_y} = \dot{\epsilon}, \quad (2.39)$$

$$\Omega_{IB_z} = \dot{\eta} \cos(\epsilon), \quad (2.40)$$

for the inner gimbal. Therefore, the resultant vectors can be expressed as

$$\boldsymbol{\Omega}_{OB} = \begin{bmatrix} 0 \\ 0 \\ \dot{\eta} \end{bmatrix}, \quad (2.41)$$

$$\boldsymbol{\Omega}_{IB} = \begin{bmatrix} -\dot{\eta}\sin(\epsilon) \\ \dot{\epsilon} \\ \dot{\eta}\cos(\epsilon) \end{bmatrix}. \quad (2.42)$$

2.11.2.2 Angular Acceleration

The angular acceleration is obtained by taking the first derivative of the velocity vector $\boldsymbol{\Omega}$ with respect to time. The angular acceleration of frame F_1 in relation to F_2 is expressed as $\boldsymbol{\alpha}_{F_1 F_2} = [\alpha_{F_1 F_2 x}, \alpha_{F_1 F_2 y}, \alpha_{F_1 F_2 z}]^T$. Therefore, the angular acceleration vectors for our system are:

$$\alpha_{OB_x} = 0, \quad (2.43)$$

$$\alpha_{OB_y} = 0, \quad (2.44)$$

$$\alpha_{OB_z} = \ddot{\eta}, \quad (2.45)$$

and

$$\alpha_{IB_x} = -(\ddot{\eta}\sin(\epsilon) + \dot{\eta}\dot{\epsilon}\cos(\epsilon)), \quad (2.46)$$

$$\alpha_{IB_y} = \ddot{\epsilon}, \quad (2.47)$$

$$\alpha_{IB_z} = \ddot{\eta}\cos(\epsilon) - \dot{\eta}\dot{\epsilon}\sin(\epsilon). \quad (2.48)$$

2.11.3 Friction Model

Friction between moving components represents a significant and often nonlinear dynamic effect within mechanical systems, particularly prevalent in gimbal platforms. Friction is inherently present in the bearings supporting both the inner and outer gimbal axes, and its influence becomes particularly pronounced during changes in the direction of rotation.

For modeling purposes, the total friction force (or torque) can be broadly categorized into two primary types: viscous friction and Coulomb friction. The overall friction torque

(T_{friction}) acting on the system is thus expressed as the sum of these two components:

$$T_{\text{friction}} = T_{\text{viscous}} + T_{\text{coulomb}},$$

where T_{viscous} denotes the viscous friction torque and T_{coulomb} represents the Coulomb friction torque. Viscous friction is characterized by its proportionality to the relative velocity between the moving surfaces, while Coulomb friction is nonlinear in nature and is primarily based on the friction between two objects resulting from the applied normal force. In Ramírez (2016), these friction components are mathematically formulated as:

$$T_{\text{viscous}} = k_{\text{vf}}\Omega, \quad (2.49)$$

where k_{vf} is the viscous friction coefficient between the moving parts in the gimbal system, and Ω represents the angular velocity rotating body (inner or outer gimbal), and

$$T_{\text{coulomb}} = k_{\text{cf}}\text{sign}(\Omega), \quad (2.50)$$

where k_{cf} is the Coulomb friction coefficient, and $\text{sign}(\Omega)$ is the sign function of the angular velocity, ensuring that the friction torque opposes the direction of motion.

This detailed modeling of friction results in a more realistic dynamic behavior of the gimbal system, allowing for a more accurate simulation and subsequent controller design that accounts for these inherent nonlinearities.

2.11.4 Gimbal Dynamics

To obtain the equations describing the gimbal's dynamics, we will employ the Newton-Euler mechanics. This approach, based on applying Newton's laws for translation and Euler's laws for rotation to each rigid body in the system, is widely used in the modeling of mechatronic and robotic systems, as detailed in works such as Boiffier (1998).

The choice of the Newton-Euler method is justified by its ability to provide a direct representation of the relationships between applied forces/torques and the resulting motion, making it particularly useful for systems with rotational joints like gimbals. To simplify the derivation of the equations and focus on the central aspects of tracking control, premises are adopted regarding mass distribution and rotation axes, such as homogeneous mass distribution, which is the assumption that mass is homogeneously distributed within the gimbals and the alignment with principal axes of inertia, ensuring that the axes of rotation coincide exactly with the principal axes of inertia of each link. These simplifications implies that products of inertia are null, resulting in diagonal inertia tensors. These simplifications neglect cross-coupling terms in the inertia matrix, which would otherwise

arise from mass unbalance or misalignment of principal axes, thereby simplifying the dynamic equations. Such an approach is common in initial models to reduce complexity and is explored in works such as Poyrazoğlu (2017) and Ramírez (2016), which also use this premise to simplify the modeling of gimbal dynamics. Another assumption is the fixed base, in which the base is considered fixed and represents the system's inertial reference frame. This means that no movements or external disturbances act upon the base, simplifying the kinematic and dynamic equations by eliminating terms related to the supporting platform's dynamics.

These premises allow for a clearer derivation of the fundamental equations, which will serve as the basis for the subsequent incorporation of specific nonlinearities of the system and the actuator, in line with the objective of this work.

2.11.4.1 Inner Gimbal

The angular momentum of the inner gimbal about the pivot point P relative to the inner gimbal frame is

$$\mathbf{H}_{I,P}^I = \mathbb{I}_P^I \boldsymbol{\Omega}_{IB}. \quad (2.51)$$

Here, $\mathbb{I}_P^I$ represents the inertia tensor of the decoupled inner gimbal, calculated about the pivot point, and $\boldsymbol{\Omega}_{IB}$ denotes the inner gimbal's angular velocity vector relative to the Earth frame. The total moment exerted on the inner gimbal is obtained from the first-order time derivative of its angular momentum. Building upon calculations presented in Boiffier (1998), Euler's equation for the inner gimbal, specifically about its pivot point, is thus formulated as:

$$\mathbf{M}_{OI} = \frac{d\mathbf{H}_{I,P}^O}{dt} = \mathbb{I}_P^I \boldsymbol{\alpha}_{IB} + \boldsymbol{\Omega}_{IB} \times \mathbf{H}_{I,P}^I. \quad (2.52)$$

In equation (2.52), $\mathbf{H}_{I,P}^O$ is the angular momentum of the inner gimbal about the pivot point P relative to the outer gimbal frame, and $\boldsymbol{\alpha}_{IB}$ is the inner gimbal's angular acceleration vector. Using the premises stated in the beginning of this section, the inertia tensor and, therefore, the vector representing the total moment exerted on the inner gimbal can be expressed as

$$\mathbb{I}_P^I = \begin{bmatrix} I_x^I & 0 & 0 \\ 0 & I_y^I & 0 \\ 0 & 0 & I_z^I \end{bmatrix}, \quad (2.53)$$

$$\mathbf{M}_{OI} = \begin{bmatrix} I_x^I \alpha_{IBx} + (I_z^I - I_y^I) \Omega_{IBy} \Omega_{IBz} \\ I_y^I \alpha_{IBy} + (I_x^I - I_z^I) \Omega_{IBx} \Omega_{IBz} \\ I_z^I \alpha_{IBz} + (I_y^I - I_x^I) \Omega_{IBx} \Omega_{IBy} \end{bmatrix}. \quad (2.54)$$

Including the effects of friction torque in the total moment, and considering that torque

is transferred from outer gimbal to the inner gimbal along the y axis, therefore neglecting the components in x and z axes, the resultant vector is now

$$\begin{bmatrix} 0 \\ T_{m_I} - T_{\text{Friction}_{OI}} \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ I_y^I \alpha_{IB_y} + (I_x^I - I_z^I) \Omega_{IB_x} \Omega_{IB_z} \\ 0 \end{bmatrix}, \quad (2.55)$$

which gives us the equation for elevation torque for the angular momentum in the y axis for the inner gimbal:

$$T_{m_I} - T_{\text{Friction}_{OI}} = I_y^I \alpha_{IB_y} + (I_x^I - I_z^I) \Omega_{IB_x} \Omega_{IB_z}. \quad (2.56)$$

In equation (2.56), T_{m_I} is the torque transmitted from the outer gimbal to the inner gimbal by an electrical motor, and $T_{\text{Friction}_{OI}}$ is the friction torque between outer and inner gimbals.

2.11.4.2 Outer Gimbal

The angular momentum of the outer gimbal about the pivot point P relative to the outer gimbal frame is

$$\mathbf{H}_{O,P}^O = \mathbb{I}_P^O \boldsymbol{\Omega}_{OB} + \mathbb{T}_{IO} \mathbf{H}_{I,P}^I. \quad (2.57)$$

Here, $\mathbb{I}_P^O$ represents the inertia tensor of the decoupled outer gimbal, calculated about the pivot point, and $\boldsymbol{\Omega}_{OB}$ denotes the outer gimbal's angular velocity vector relative to the Earth frame. Notice that this equation already consider the influence of the inner gimbal in the outer gimbal's angular momentum. Applying the same considerations used in equation (2.52), Euler's equation of the outer gimbal about the pivot point is expressed as

$$\mathbf{M}_{BO} + \mathbf{M}_{IO} = \frac{d\mathbf{H}_{O,P}^B}{dt}, \quad (2.58)$$

$$\frac{d\mathbf{H}_{O,P}^B}{dt} = \mathbb{I}_P^O \boldsymbol{\alpha}_{OB} + \dot{\mathbb{T}}_{IO} \mathbb{I}_P^I \boldsymbol{\Omega}_{IB} + \mathbb{T}_{IO} \mathbb{I}_P^I \boldsymbol{\alpha}_{IB} + \boldsymbol{\Omega}_{OB} \times (\mathbb{I}_P^O \boldsymbol{\Omega}_{OB} + \mathbb{T}_{IO} \mathbb{I}_P^I \boldsymbol{\Omega}_{IB}), \quad (2.59)$$

where $\mathbf{H}_{O,P}^B$ is the angular momentum of the outer gimbal about the pivot point P relative to the base frame. By expanding the terms in equation (2.59), and under the assumption that torque is transferred from base to the outer gimbal along the z axis, thus $M_{IO_x} = M_{IO_y} = M_{BO_x} = M_{BO_y} = 0$, the resultant vectors for M_{BO} and M_{IO} are:

$$\mathbf{M}_{IO} = \begin{bmatrix} 0 \\ 0 \\ T_{\text{Friction}_{IO}} \end{bmatrix}, \quad \mathbf{M}_{BO} = \begin{bmatrix} 0 \\ 0 \\ T_{m_O} - T_{\text{Friction}_{BO}} \end{bmatrix}.$$

In this case, $T_{\text{Friction}_{BO}}$ indicate the friction torques between the base and the outer gimbal, while T_{m_O} being the motor torque applied to the outer gimbal. $T_{\text{Friction}_{IO}}$ is the influence that the friction torque between inner and outer gimbals exerts on outer gimbal's motion, and will be considered to be zero for simplification purposes. Due to the alignment of the rotation axes of the base and the outer gimbal along the z axis, the sum of T_{m_O} and $T_{\text{Friction}_{BO}}$ is directly transmitted to the outer gimbal, whereas $T_{\text{Friction}_{IO}}$ is a friction torque transferred indirectly. Therefore, the equation of motion for the outer gimbal on the z axis is given by

$$\begin{aligned} T_{m_O} - T_{\text{Friction}_{BO}} = & I_z^O \alpha_{OB_z} + I_z^I \alpha_{IB_z} \cos(\epsilon) + I_x^I \alpha_{IB_x} \sin(\epsilon) \\ & - I_x^I \Omega_{IB_x} \Omega_{IB_y} \cos(\epsilon) + I_z^I \Omega_{IB_z} \Omega_{IB_y} \sin(\epsilon). \end{aligned} \quad (2.60)$$

3 Object Tracking System

This Chapter describes the proposed methodology to implement an image-based gimbal feedback control model for a two-degrees-of-freedom gimbal platform combined with a computer vision system. The solution can be divided into four different stages: Image Acquisition, Object Detection, State Estimation, and Gimbal Control.

3.1 Architecture

The overall architecture of the proposed system is designed to facilitate real-time object tracking and aiming, integrating several distinct yet interconnected stages. Figure 3.1 illustrates the comprehensive flow of data and the interaction between each primary component, from initial image acquisition to the final aiming mechanism.

The simulated Camera is responsible for capturing a sequence of input image frames, serving as the primary data source for the entire system. The raw image data from the camera is then fed into the Object Detection algorithm.

The Object Detection algorithm processes the incoming image frames to identify and localize the target object. As previously discussed, this module outputs the coordinates of the detected object within each frame. These detected coordinates, though crucial, are prone to noise and inconsistencies.

To address these challenges, the output from the Object Detection stage is passed to the State Estimation module, specifically to the Kalman filter. The Kalman filter plays a vital role by utilizing the noisy coordinate data to estimate the true state of the object, effectively reducing noise, smoothing the signal, and mitigating the effects of data loss or delay. The output of the Kalman filter component is a refined, smoothed trajectory of the target object.

The estimated object state from the Object Tracking module then serves as input for the Motor Controller. This controller translates the object position into a voltage signal for the DC servomotor in the Gimbal Model. The result is an adjustment in the camera's orientation to keep the target object centered in the image. This creates a closed-

loop system where the camera's input is processed to control its own orientation, ensuring continuous and accurate tracking of the target object. This integrated architecture ensures a seamless flow from visual input to precise mechanical response, enabling robust object tracking and aiming capabilities.

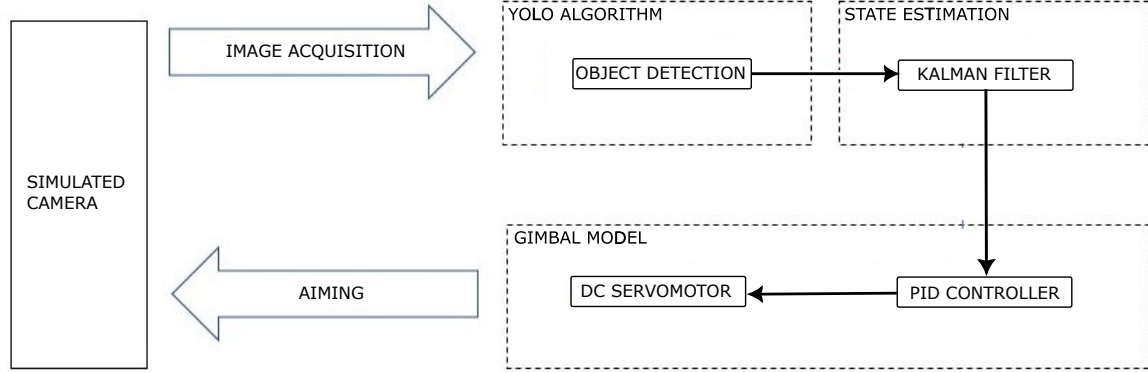


FIGURE 3.1 – Proposed architecture for the system.

3.2 Image Acquisition

Images are acquired by a camera coupled in a gimbal, resulting in a configuration similar to the what was presented in Section 2.11.1. Every frame captured will serve as input for the object detection algorithm YOLOv8, and the response from the gimbal control will adjust the camera's pointing direction.

A virtual world is created using Simulink[®] 3D Animation[™] Toolbox to simulate the camera behavior and image capturing. Further, frames are generated using *Simulation 3D Camera* block. Image 3.2 shows this configuration.

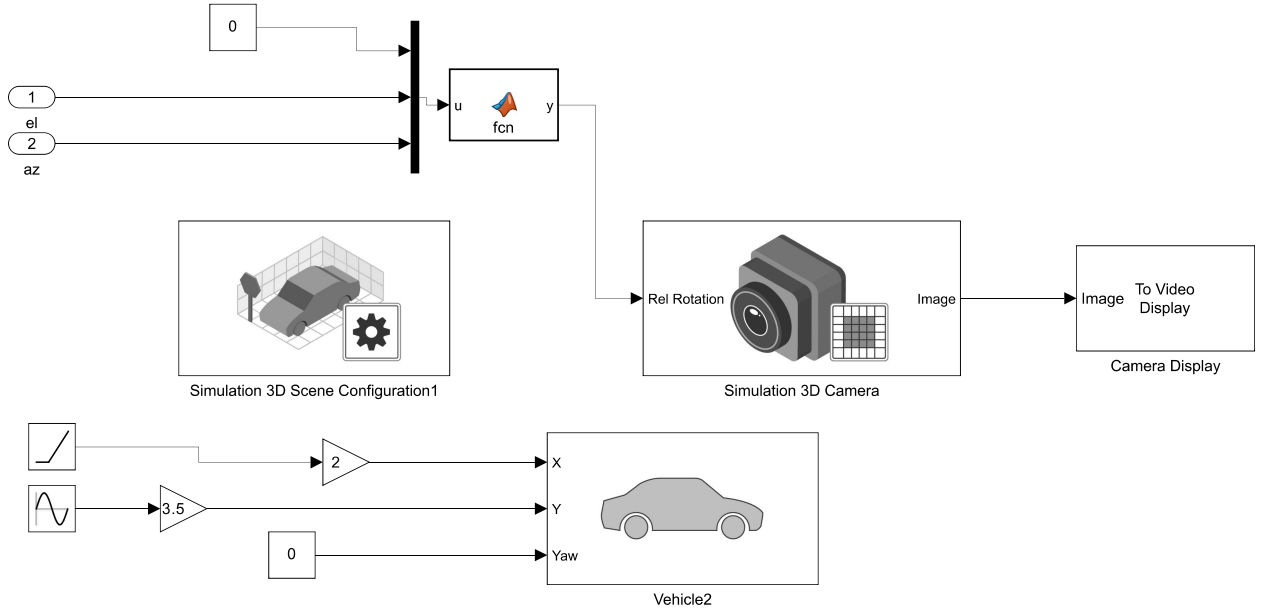


FIGURE 3.2 – Image acquisition model.

The *Simulation 3D Camera* block maps objects from a three-dimensional space to a two-dimensional space, utilizing the ideal pinhole camera model. The pinhole camera model can be understood as the central projection of points in space onto a plane. According to Hartley e Zisserman (2004), under the pinhole camera model, a point in space with coordinates $\mathbf{X} = [X, Y, Z]^T$ is mapped to the point on the image plane where a line joining the point $\mathbf{X}$ to the centre of projection meets the image plane. Figure 3.3 illustrates this process, with $\mathbf{C}$ being the camera centre and $\mathbf{p}$ the principal point. The images generated have dimensions of 1280 pixels in width and 720 pixels in height.

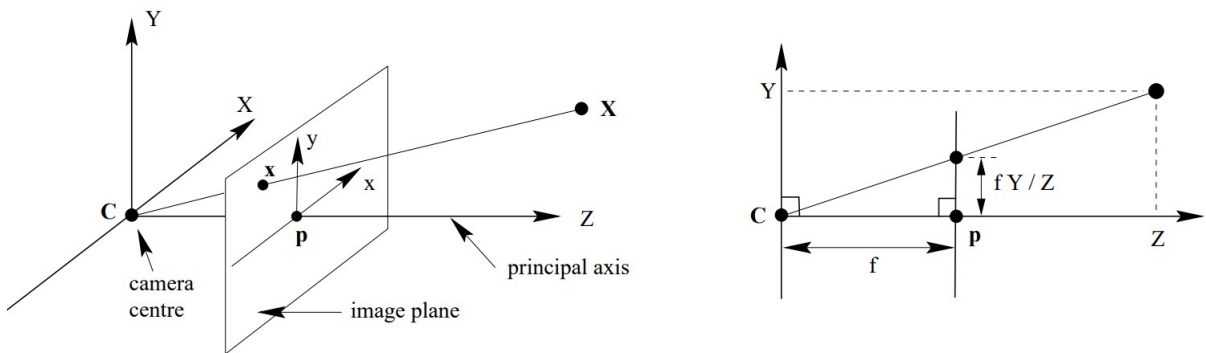


FIGURE 3.3 – Pinhole camera geometry. Extracted from Hartley e Zisserman (2004).

The generated frames are displayed in a window using the Camera Display block. The result is shown in Figure 3.4.

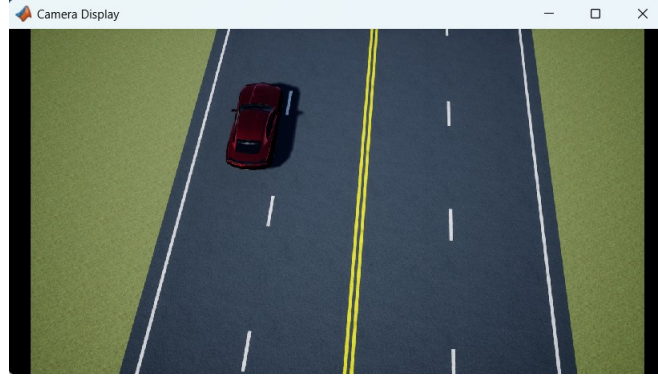


FIGURE 3.4 – Resultant frame.

With the virtual world and camera correctly modeled, the frames are then recorded using a virtual camera that is generated using the software OBS Studio 31.0.3, which will then be accessed by the object detection algorithm.

3.3 Object Detection Algorithm

The recorded images serve as input to the multi-object detection algorithm YOLOv8, implemented in *Python* programming language using Ultralytics, a computer vision package for object detection and image segmentation models, with a strong focus on the You Only Look Once (YOLO) family of models. The algorithm receives each frame as input and return arrays containing information about the detected object, such as its class, position of the bounding box in the image, and more. We are interested in obtaining the center of the bounding box, represented by a vector $\mathbf{v} = [x, y]$, where x is the position of the object in the horizontal axis of the image plane in pixels, and y is the position of the object in the vertical axis of the image plane in pixels. Every frame received by YOLOv8 is resized from 1280x720 to 640x480 pixels. Figure 3.5 shows the pixel range for a given frame.

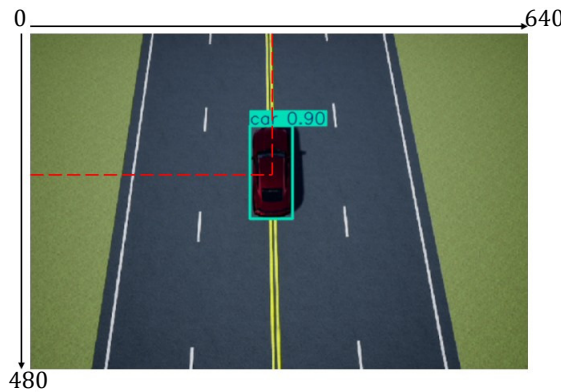


FIGURE 3.5 – Pixel range for each frame.

This information is then sent to the Simulink model using User Datagram Protocol (UDP) protocol. UDP is a connectionless communication protocol that operates at the transport layer of the Internet Protocol suite. This protocol does not ensure data delivery or order, but ensure low-latency communication, which makes it interesting for data streaming applications (GATIMU *et al.*, 2020).

3.4 State Estimation

The object detection algorithm is responsible for recognizing the target object in the input image frame sequence and return its coordinates in every image frame of the acquired sequence. Even though the system is capable of detecting multiple objects, only one object will serve as input for the state estimator algorithm. The raw coordinate data obtained from the object detection algorithm, despite its precision, is inherently susceptible to various forms of noise. This noise can originate from environmental factors, sensor limitations, or even transient occlusions, leading to inconsistencies and abrupt fluctuations in the reported object positions. The Kalman filter is being employed to smooth the received measurements.

The Kalman filter was designed to track a 2D object's position (x, y) and velocity (v_x, v_y) . Its state vector comprises four elements $\hat{\mathbf{x}} = [x, y, v_x, v_y]^T$. The state transition matrix A models constant velocity, implying that, in the absence of external forces, the object's velocity remains constant, leading to a linear change in its position. However, the inclusion of a non-zero input vector u and the input matrix B further integrates a known, constant acceleration of $[1; 1]$ in both the x and y directions into the model's prediction. The specific matrices are

$$A = \begin{pmatrix} 1 & 0 & T & 0 \\ 0 & 1 & 0 & T \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} T^2/2 & 0 \\ 0 & T^2/2 \\ T & 0 \\ 0 & T \end{pmatrix}, \quad u = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

The measurement model is defined by the measurement matrix C , which indicates that only the object's 2D position (x, y) is directly observed by the sensor (YOLOv8 algorithm). The velocities are not measured, but are estimated by the filter. Thus,

$$C = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

The time step is set to $T = 0.002$, implying a high sampling rate, which is beneficial for tracking rapidly changing dynamics, but demands a careful scaling of the noise parameters.

The performance of the Kalman filter heavily relies on its noise parameters. The Process Noise Covariance Q matrix accounts for the uncertainty in the object's motion model and any unmodeled accelerations or disturbances. It is derived from $\sigma_{acc} = 2$, making the filter less responsive to changes in velocity, effectively causing it to trust its motion model more, thus ignoring unpredictable movements. The Model Covariance Noise Q is given by

$$Q = B \cdot M \cdot B^T,$$

where

$$M = \begin{pmatrix} \sigma_{acc}^2 & 0 \\ 0 & \sigma_{acc}^2 \end{pmatrix}.$$

Conversely, the Measurement Noise Covariance R matrix represents the expected noise in the incoming YOLOv8 position measurements. The standard deviation $\sigma_{cam} = 1000$ indicates that the filter considers the input signal to be extremely noisy. This high σ_{cam} value means the filter will place very little weight on the instantaneous YOLOv8 measurements, instead relying heavily on its internal predictions and previous estimates to produce a smooth output. Measurement Noise Covariance matrix R is written as

$$R = \begin{pmatrix} \sigma_{cam}^2 & 0 \\ 0 & \sigma_{cam}^2 \end{pmatrix}.$$

Using the methodology proposed in Section 2.10 associated with the described model, the result for the filtering process is given by Table 3.1.

TABLE 3.1 – Resultant equations for the Kalman filter recursive process.

Description	Equation
Kalman Gain	$K_k = P'_k C^T (C P'_k C^T + R)^{-1}$
Update Estimate	$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}'_k + K_k (z_k - C \hat{\mathbf{x}}'_k)$
Update Covariance	$P_k = P'_k - K_k C P'_k$
Project into $k + 1$	$\hat{\mathbf{x}}_{k+1} = A \hat{\mathbf{x}}_k$ $P_{k+1} = A P_k A^T + Q$

Figure 3.6 shows the comparison between the received signal from YOLOv8 and the output from the Kalman filter.

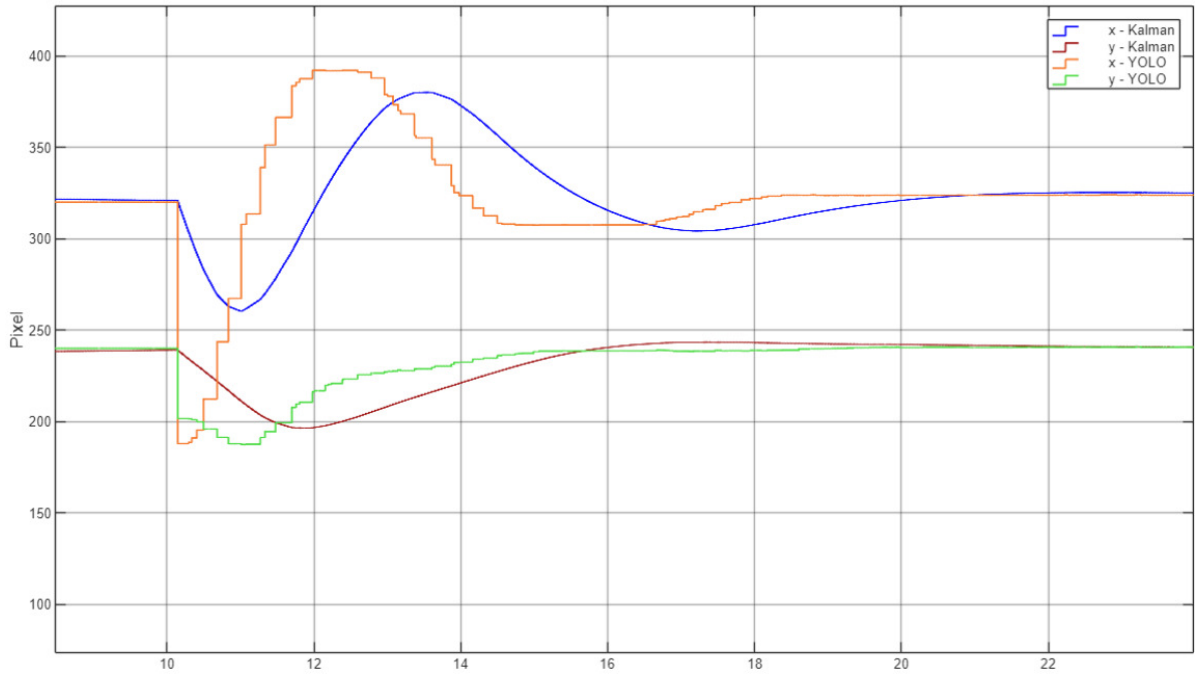


FIGURE 3.6 – Kalman filter applied to the YOLOv8's detections.

3.4.1 Post Filter Processing

After the Kalman filter, a processing step is responsible for preparing the signal for the controller. The purpose is to shape the signal according to the camera's field-of-view. Firstly, a gain is applied to rescale the signal to 1280x720 pixels, matching the simulated camera's output dimensions. Then, using the camera's intrinsic matrix associated with homogeneous coordinates, we normalize the received vector and translate the information to angular position. The first step is to convert the raw 2D pixel coordinates (u, v) from the image plane into normalized image coordinates (x_n, y_n) in the camera's coordinate system. This process effectively inverts the camera's intrinsic matrix (focal length, principal point), as represented in equation 3.1.

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}. \quad (3.1)$$

The intrinsic matrix (HARTLEY; ZISSERMAN, 2004) is represented as

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad (3.2)$$

where f_x and f_y represents the focal length in x and y directions, respectively, and c_x and c_y represent the coordinates of the principal point. With the normalized vector, a gain

is applied to generate a signal that represents the object's position with relation to the camera's field-of-view for each axis, in radians.

3.5 Gimbal Model

The dynamics of the two-degrees-of-freedom gimbal is modeled using Simulink, and the communication with the object detection algorithm will result in the system's response to the input images, ensuring that the camera is pointing to the desired target. The model is reduced to the elevation and azimuth axes, where in each case the respective inertia is represented. For each axis, there is a controlled electrical motor responsible for the torque that rotates the mass. The model is implemented via transfer function.

3.5.1 Rotational Axes

The rotational axes are divided into 3 main blocks: the Controller, the DC motor and the Gimbal (for elevation and azimuth). Starting with the Inner Gimbal, the rotational movement expressed in equation (3.3) is used. Given that the only source of torque is the electrical motor, the equation can be rewritten as

$$T_{m_I} = I_y^I \alpha_{IB_y} + (I_x^I - I_z^I) \Omega_{IB_x} \Omega_{IB_z} + T_{\text{Friction}_{OI}}. \quad (3.3)$$

This express the torque needed from the motor to rotate the mass. As mentioned in Section 2.11.4, some considerations are adopted to simplify the model. By eliminating the effects from Cross-Coupling, the equation is now

$$T_{m_I} = I_y^I \alpha_{IB_y} + T_{\text{Friction}_{OI}}. \quad (3.4)$$

The Coulomb friction model is an distrubance in the torque signal due to the non linear nature of this phenomenon. Therefore, the final equation for the inner gimbal rotational dynamic becomes a linear function expressed as

$$T_{m_I} = I_y^I \alpha_{IB_y} + k_{vf} \Omega_{IB_y}. \quad (3.5)$$

The same process is applied to the outer gimbal, resulting in the equation

$$T_{m_O} = I_z^O \alpha_{OB_z} + k_{vf} \Omega_{OB_z}. \quad (3.6)$$

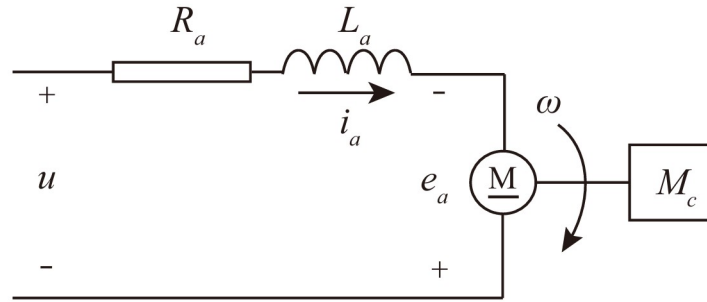
Table 3.2, extracted from Ramírez (2016) shows the inertial parameters for the inner and outer gimbals.

TABLE 3.2 – Gimbal's Inertial Parameters.

Parameter	Value
I_x^I	$1.68 \times 10^{-4} \text{ (kg} \cdot \text{m}^2\text{)}$
I_y^I	$1.78 \times 10^{-4} \text{ (kg} \cdot \text{m}^2\text{)}$
I_z^I	$1.16 \times 10^{-4} \text{ (kg} \cdot \text{m}^2\text{)}$
I_x^O	$2.62 \times 10^{-3} \text{ (kg} \cdot \text{m}^2\text{)}$
I_y^O	$2.47 \times 10^{-3} \text{ (kg} \cdot \text{m}^2\text{)}$
I_z^O	$4.66 \times 10^{-4} \text{ (kg} \cdot \text{m}^2\text{)}$

3.5.2 Electrical Motors

The electrical motors are the source of torque in both axes, azimuth and elevation. These motors will be responsible for converting the voltage signal into a torque, and are modeled using the equivalent circuit diagram of DC servo motors, as shown in figure 3.7.

FIGURE 3.7 – Equivalent circuit diagram of DC servo motor. Extracted from Zhou *et al.* (2018).

In this representation, R_a is the resistance of the armature coil, L_a is the armature inductance, i_a is the armature current, e_a is the counter electromotive force, u is the applied voltage, ω (Ω in our system) is the rotational speed, and M_c is the load torque. Using Kirchhoff's Voltage law, this circuit can be mathematically expressed as

$$u(t) = R_a i_a(t) + L_a \dot{i}_a(t) + e_a(t). \quad (3.7)$$

The counter electromotive force is the relation between the counter electromotive constant, K_e , and the rotational speed, that is the first derivative of the angular position in relation to time, $\dot{\theta}(t)$. This allows us to rewrite equation (3.7) into

$$u(t) = R_a i_a(t) + L_a \dot{i}_a(t) + K_e \dot{\theta}(t). \quad (3.8)$$

Lastly, the equation for the load torque is given as follows:

$$M_c(t) = C_m i_a(t), \quad (3.9)$$

where C_m is the torque constant and $i_a(t)$ is the armature current. The electrical motors are modeled using the parameters of the FAULHABER® 1524T012SR DC motor (Figure 3.8). Table 3.3 shows the parameters for this DC motor.



FIGURE 3.8 – FAULHABER® 1524T012SR.

TABLE 3.3 – FAULHABER® 1524T012SR parameters

Parameter	Value
Nominal Voltage	12 [V]
Armature resistance	0.674 [Ω]
Output Power	1.75 [W]
Max efficiency	0.76
No-load speed	9900 [rpm]
No-load current	0.011 [A]
Friction torque	0.13 [mNm]
Speed constant	840 [rpm/V]
Torque constant	11.4 [mNm/A]
Armature Inductance	250 [μ H]
Rotor inertia	0.65 [g cm ²]
Angular acceleration	100 $\times$ 10 ³ [rad/s ²]
Weight	21 [g]
Stall torque	6.92 10 ⁻³ [Nm]

Associated with the electrical motor, the parameters of the encoder are given in Table 3.4. Encoders are vital components in DC servomotor systems, providing crucial feedback for precise control of position, speed, and direction. They essentially translate the motor's rotational motion into electrical signals that a controller can interpret. In this work, the parameters of FAULHABER® IE2-512 are being used.

TABLE 3.4 – FAULHABER® IE2-512 parameters.

Parameter	Value
Pulses per resoluton	512
Max frequency	160×10^3 [Hz]
Inertia	0.09 [g·cm ²]
Phase shift	90 [°]

3.5.3 Transfer Functions

Using equations (3.5), (3.8) and (3.9), the transfer function that relates input voltage to motor torque ($T_{m_I}(s)/U(s)$) and motor torque to angular position ($\Theta(s)/T_{m_I}(s)$) can be obtained. To give more realism to these functions, the viscous friction is augmented with the motor's bearings friction and the rotational inertia of the motor and encoder are also considered. Thus, assuming zero initial conditions, the Laplace transform of these equations yields:

- From (3.5):

$$T_{m_I}(s) = I_y^I s^2 \Theta(s) + k_{vf} s \Theta(s)$$

$$T_{m_I}(s) = (I_y^I s + k_{vf}) s \Theta(s) \quad (3.10)$$

- From (3.8):

$$U(s) = R_a I_a(s) + L_a s I_a(s) + K_e s \Theta(s)$$

$$U(s) = (R_a + L_a s) I_a(s) + K_e s \Theta(s) \quad (3.11)$$

- From (3.9):

$$T_{m_I}(s) = C_m I_a(s) \quad (3.12)$$

To derive $\frac{T_{m_I}(s)}{U(s)}$, we first express $I_a(s)$ and $s\Theta(s)$ in terms of $T_{m_I}(s)$ using equations (3.12) and (3.10):

$$I_a(s) = \frac{T_{m_I}(s)}{C_m} \quad (3.13)$$

$$s\Theta(s) = \frac{T_{m_I}(s)}{I_y^I s + k_{vf}} \quad (3.14)$$

Now, by using (3.13) and (3.14) into (3.11) (in Laplace domain):

$$U(s) = (R_a + L_a s) \left(\frac{T_{m_I}(s)}{C_m} \right) + K_e \left(\frac{T_{m_I}(s)}{I_y^I s + k_{vf}} \right)$$

Factoring $T_{m_I}(s)$ from the right-hand side:

$$U(s) = T_{m_I}(s) \left[\frac{R_a + L_a s}{C_m} + \frac{K_e}{I_y^I s + k_{vf}} \right]$$

Applying algebraic manipulations, we can obtain the desired transfer function

$$\frac{T_{m_I}(s)}{U(s)} = \frac{C_m(I_y^I s + k_{vf})}{L_a I_y^I s^2 + (R_a I_y^I + L_a k_{vf})s + (R_a k_{vf} + C_m K_e)}, \quad (3.15)$$

where $T_{m_I}(s)$ represents the Laplace transform of the motor torque, $U(s)$ denotes the Laplace transform of the input voltage to the motor, and I_y^I is the moment of inertia of the inner gimbal about its y-axis. The viscous friction coefficient is denoted by k_{vf} , while R_a and L_a represent the armature resistance and armature inductance of the motor, respectively. C_m is the motor torque constant, K_e is the back-EMF constant, and s is the Laplace operator.

The transfer function from motor torque to angular position ($\Theta(s)/T_{m_I}(s)$) is directly obtained from equation (3.5):

$$\frac{\Theta(s)}{T_{m_I}(s)} = \frac{1}{(I_y^I s + k_{vf})s} \quad (3.16)$$

Similarly, the transfer functions for the outer gimbal are obtained:

$$\frac{T_{m_O}(s)}{U(s)} = \frac{C_m(I_z^O s + k_{vf})}{L_a I_z^O s^2 + (R_a I_z^O + L_a k_{vf})s + (R_a k_{vf} + C_m K_e)}, \quad (3.17)$$

$$\frac{\Theta(s)}{T_{m_O}(s)} = \frac{1}{(I_z^O s + k_{vf})s} \quad (3.18)$$

Figure 3.9 illustrates the implementation in Simulink.

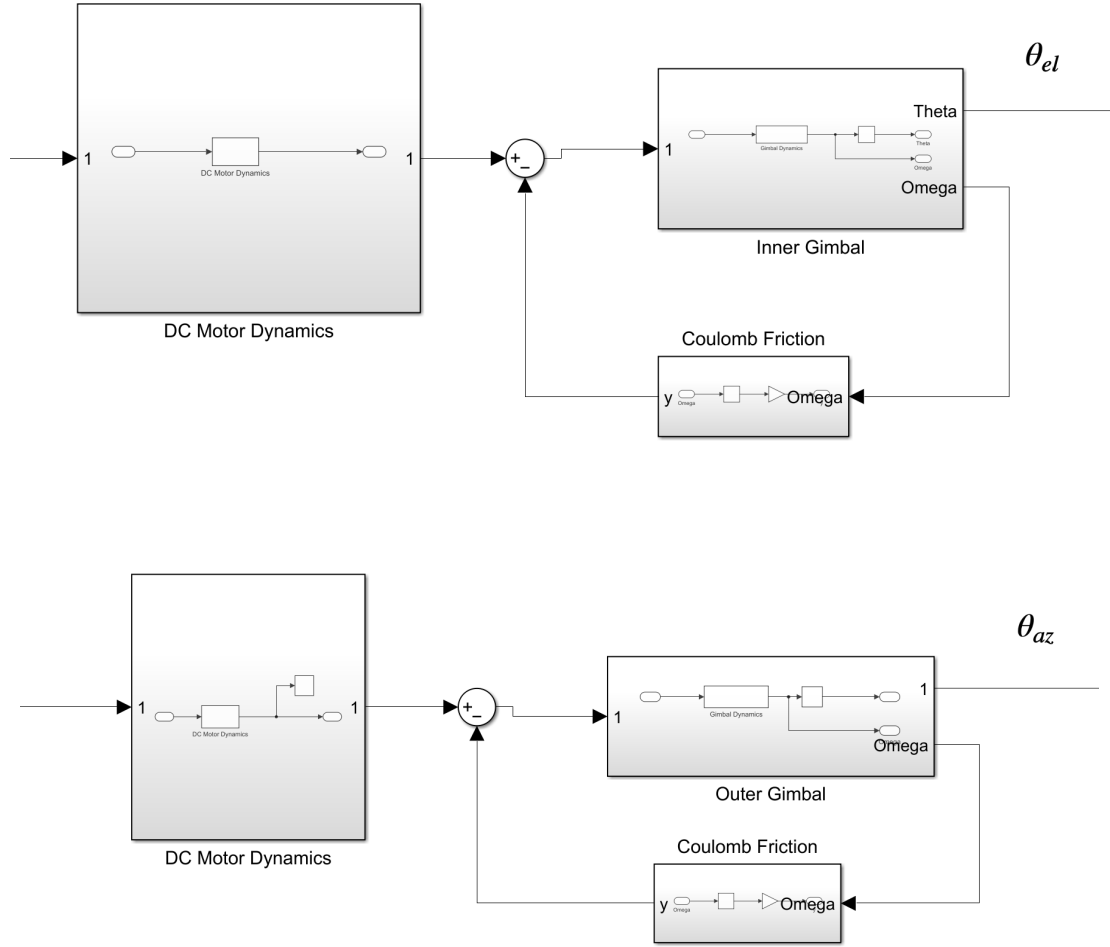


FIGURE 3.9 – Simulink model for DC servomotor and gimbals.

3.5.4 PID Controller

The Proportional-Integral-Derivative (PID) controller was chosen due to its simplicity, robustness, and effectiveness. It operates by adjusting the system's output based on three distinct components, which respond to the current error, the accumulation of past errors, and the rate of change of the error.

The transfer function of the PID controller in its standard form is expressed as:

$$C(s) = K_p + \frac{K_i}{s} + K_d s,$$

where K_p is the Proportional gain, which adjusts the controller's output proportionally to the current error. A larger K_p generally results in a faster response but can lead to excessive overshoot or instability. K_i is the Integral gain, which acts upon the integral

of the error over time. Its primary function is to eliminate steady-state error, ensuring that the system's output reaches the desired value. However, an excessive K_i can cause oscillations and sluggish response. K_d is the Derivative gain, which responds to the rate of change of the error. It anticipates future error behavior, helping to dampen oscillations, reduce overshoot, and improve system stability, but a very high K_d can make the system sensitive to noise.

The `rlocus()` function in MATLAB was used for visualizing the root locus plot of a control system, since it is a graphic tool for understanding system stability and transient response as a gain parameter varies. Analyzing the root locus plots for both the elevation and azimuth axes of the gimbal system provides insights into the system's behavior and aids in making informed decisions about controller design.

The root locus plot for the elevation axis (Figure 3.10) illustrates the paths of the closed-loop poles as the controller gain is varied. A crucial observation from this plot is whether the root locus branches remain in the left-half of the s-plane, which is indicative of system stability. The proximity of the poles to the imaginary axis and their damping ratio significantly influence the system's transient response; poles far to the left on the real axis suggest a slower, overdamped response, while complex conjugate poles closer to the imaginary axis with moderate damping indicate an underdamped, oscillatory response. Poles on the imaginary axis signify marginal stability with sustained oscillations, and poles in the right-half plane denote an unstable system with growing oscillations. For the elevation axis, this plot guides the selection of a gain that achieves desired performance specifications, such as acceptable overshoot and settling time.

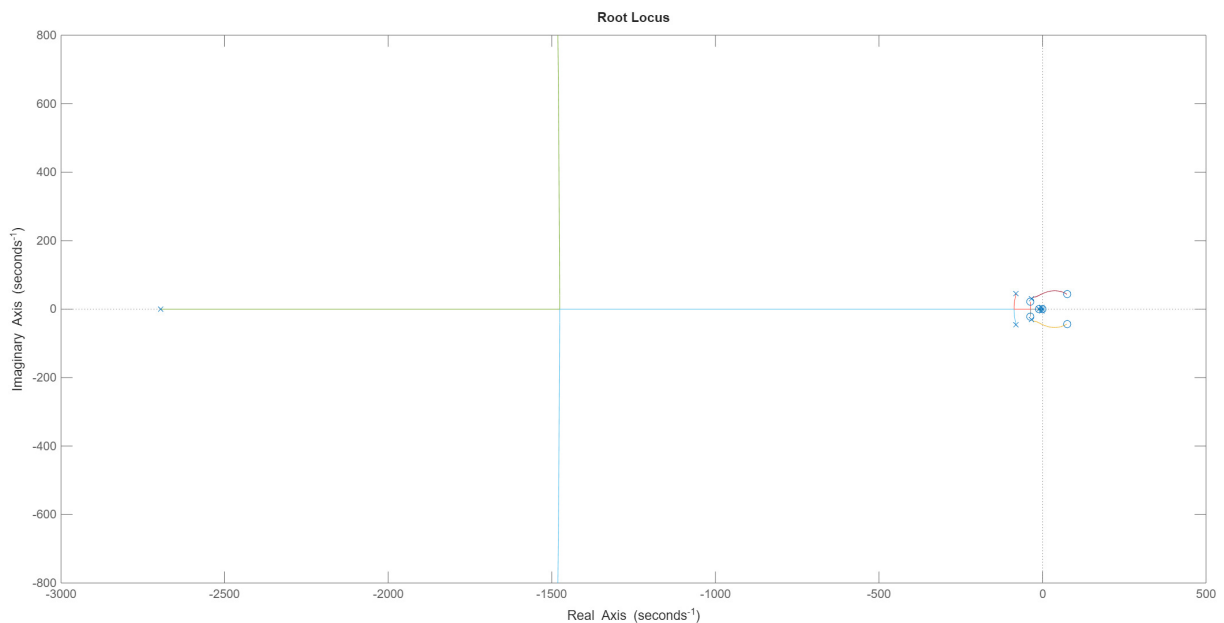


FIGURE 3.10 – Root locus plot for Elevation Axis.

Similarly, the root locus plot for the azimuth axis (Figure 3.11) offers insights into its

dynamic behavior. Given the simplifications made in the gimbal model, which often lead to analogous dynamics for both axes, one might anticipate similar characteristics to the elevation axis. However, subtle differences in inertia, friction, or controller parameters could result in distinct behaviors. By analyzing this plot, we can conclude that the poles of elevation axis system are in the left plane, meaning the system is stable.

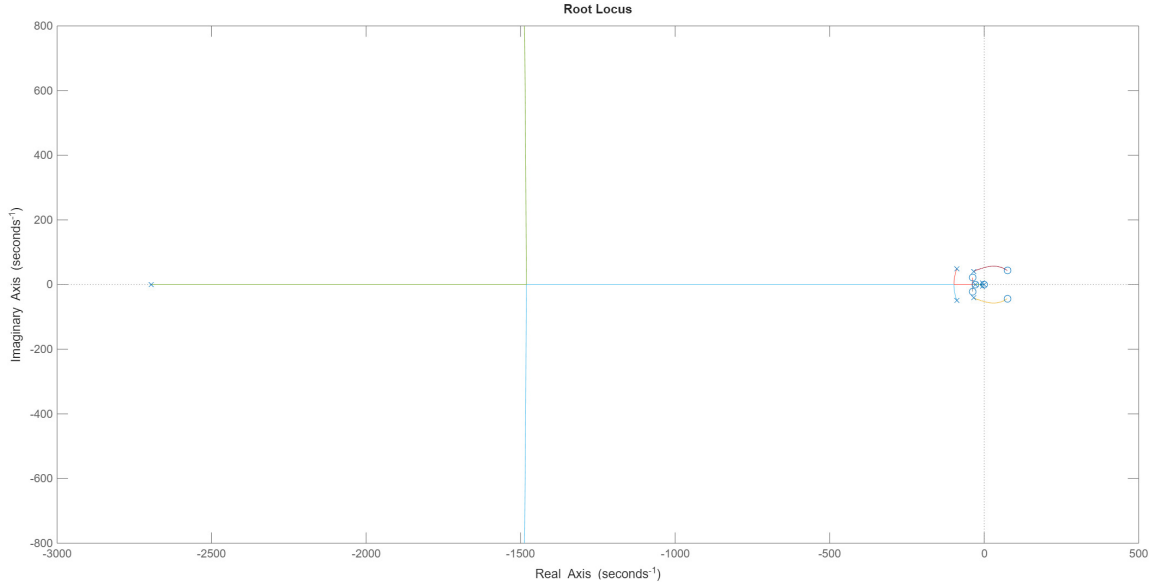


FIGURE 3.11 – Root locus plot for Azimuth Axis.

The `rlocus()` function in MATLAB is particularly useful in the design of PID controllers. Analyzing the root locus plots provides a deeper understanding of why certain gains are effective and how they influence the closed-loop pole locations. For instance, the addition of an integral term introduces a pole at the origin, which helps eliminate steady-state error but can push the locus towards instability, often requiring a compensating zero. Conversely, a derivative term introduces a zero, pulling the root locus branches to the left, thereby improving stability and speeding up the transient response. By visualizing these effects on the root locus, it becomes possible to strategically place controller poles and zeros, and then adjust the overall gain to position the closed-loop poles in desired regions of the s -plane, ultimately achieving the specified performance criteria for both the elevation and azimuth axes of the gimbal.

After obtaining the initial gains via root locus, a fine-tuning process was employed. This fine-tuning involved using the `pidentune()` function associated with repeatedly simulating the closed-loop system and observing its response to different input scenarios. Small iterative modifications to the values of K_p , K_i , and K_d were made to optimize the controller's performance regarding desired specifications, such as settling time, overshoot, and absence of steady-state error, ensuring that the system responded effectively and stably to the demands of object tracking. The final parameters for the PID in each axis are

expressed in Table 3.5.

TABLE 3.5 – PID controller parameters for each axis.

Parameter	Elevation axis	Azimuth axis
K_p	0.5	0.5
K_i	0.15	0.2
K_d	0.5	0.45

Figure 3.12 shows the controller for the elevation axis. The azimuth axis followed the same principle of implementation.

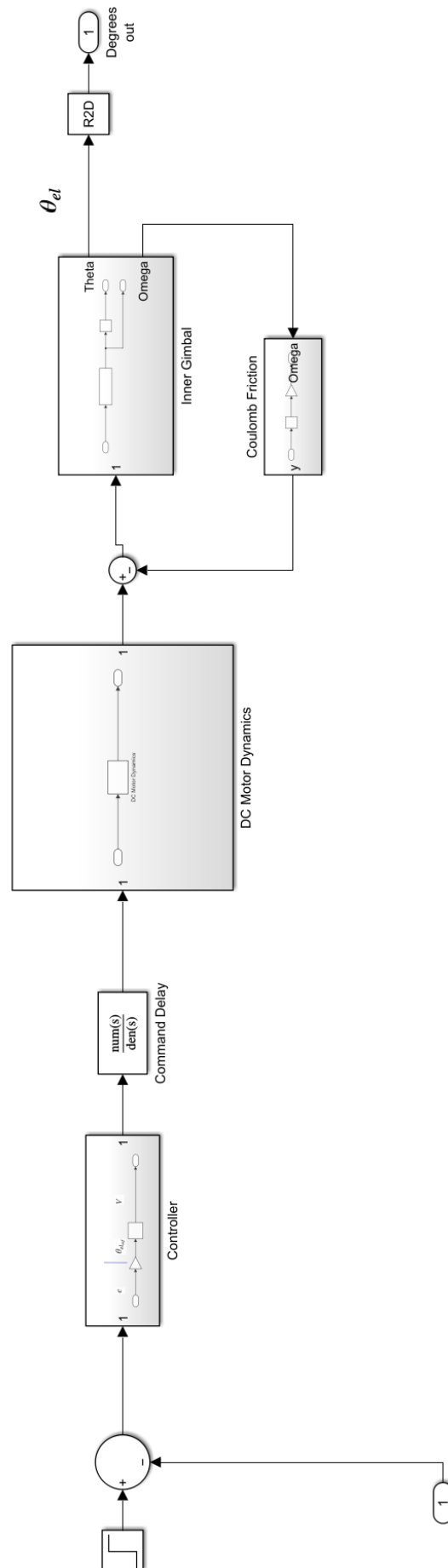


FIGURE 3.12 – PID controller implemented in Elevation Axis.

4 Results and Discussions

This chapter presents the findings of the study and engage in a detailed analysis of the results.

4.1 Training YOLO

The training was performed following the steps presented in Ultralytics documentation (ULTRALYTICS., 2024). To achieve higher performance, the algorithm was implemented in Google Colaboratory. Google Colaboratory is a web-based application for creating and sharing computational documents with code, data, and output, while also allowing access to specific GPUs.

4.1.1 Dataset

The VisDrone2019 dataset consists of 288 video clips formed by 261,908 frames and 10,209 static images, captured by various drone-mounted cameras, covering a wide range of aspects including location (taken from 14 different cities separated by thousands of kilometers in China), environment (urban and country), objects (pedestrian, vehicles, bicycles, etc.), and density (ZHU *et al.*, 2022). VisDrone comes with a set of text files (one for each image), but the annotation format must be converted to YOLO format in order to allow the training process.

4.1.2 Evaluation of Results

As stated in Lin *et al.* (2015), evaluating results on multi-class datasets is challenging due to the presence of multiple class instances per image, the non-uniform class distribution that makes simple accuracy measures inadequate, and the need for algorithm-independent evaluation measures. Mean Average Precision was used to evaluate both classification and detection in VOC2007 (EVERINGHAM *et al.*, 2010). The mAP evolution during training for our system is shown in Figure 4.1.

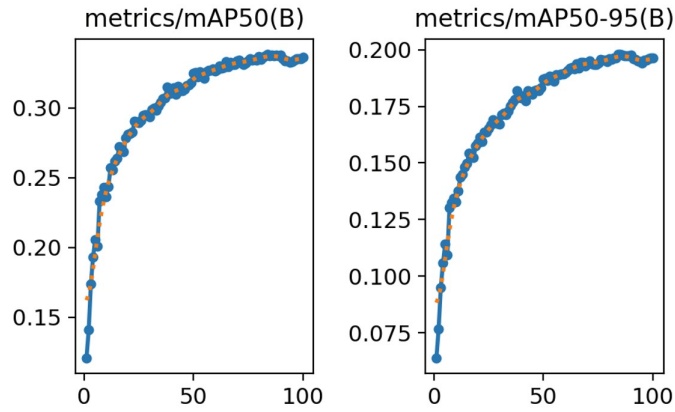


FIGURE 4.1 – mAP@50 and mAP@50-95 evolution, respectively.

The analysis of those curves shows that the model do not have the best performance. To have a deeper understanding of the problem, a breakdown of mAP for each class is presented in Table 4.1.

TABLE 4.1 – mAP@50 and mAP@50-95 for each class.

Class	mAP@50	mAP@50-95
Pedestrian	0.339	0.15
People	0.286	0.104
Bicycle	0.0804	0.0328
Car	0.756	0.519
Van	0.384	0.266
Truck	0.301	0.202
Tricycle	0.229	0.122
Awning-tricycle	0.127	0.0812
Bus	0.503	0.354
Motor	0.369	0.153

We can see that the class Car has the highest value of mAP, this may occur because the dataset has a large number of images showing cars in different conditions, which will enhance the model's capability of detecting cars. Although that is not the only reason to explain the result, it must be considered. Figure 4.2 shows the number of instances for each class.

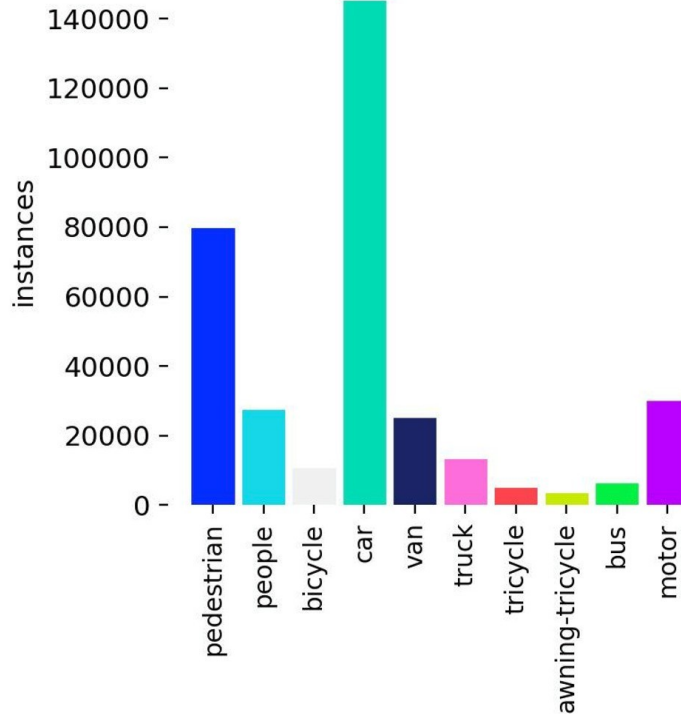


FIGURE 4.2 – Instances per class.

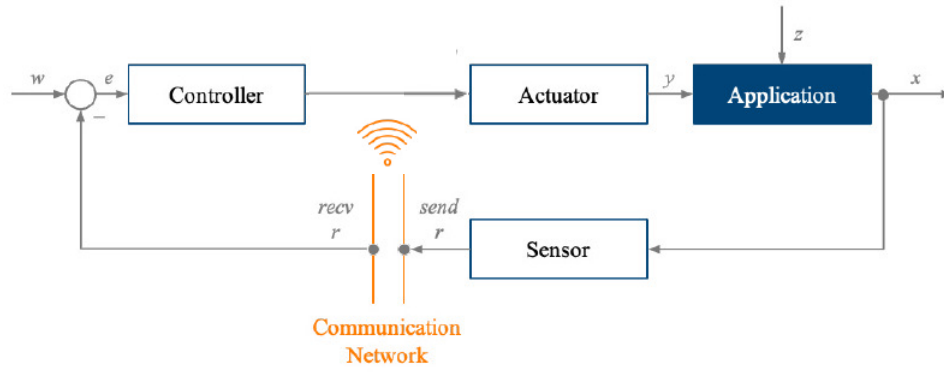
The results indicate that the dataset is imbalanced. Obaid and Nassif (2022) apply solutions to the dataset imbalance problem by utilizing Resampling approaches, and the results show an increase in the classifier’s accuracy. Changing the hyperparameters for training could also enhance the model’s performance on object detection tasks.

4.2 System Performance

In this section, we will evaluate the performance of the system with regard to characteristics of the system’s response and the communication delay from the implemented UDP protocol.

4.2.1 Latency

In computer data networks, latency is defined as the sum of temporal delays within a network. A delay is caused by the propagation and transmission of data packets within the network. To estimate and evaluate the latency, we will use the methodology proposed by Kiesel *et al.* (2022). The architecture of our network control system is shown in Figure 4.3, where the communication between the Sensor and Controller is done via UDP, and the communication between controller and actuator is considered wired, with an estimated latency of 40 milliseconds, modeled using 2nd order Padé approximation.

FIGURE 4.3 – Network communication of the control system. Extracted from Kiesel *et al.* (2022)

According to Kiesel *et al.* (2022), the time between event and action, given that each action is triggered by an event, is defined as application-perceived latency. For our system, the application-perceived latency is mathematically expressed as

$$\Delta\tau_{l,\text{application}} = \tau_{\text{sensor}} + \tau_{\text{controller}} + \tau_{\text{actuator}} + \tau_{\text{network}}, \quad (4.1)$$

where τ_{sensor} is the time in-between an event is sensed and recorded. $\tau_{\text{controller}}$ expresses the time the controller requires to analyze data from sensor and decide on a control action, which is fed back to the actuator. τ_{actuator} describes the time in-between the reception of the feedback variable and the start of the physical execution of the action. τ_{network} describes the time the network requires to transmit data from sensor to controller and from controller to actuator.

To measure τ_{sensor} and τ_{network} , the simulation was changed to generate an object in the image after 10 seconds, then it was observed when the simulation would detect it. Figures 4.4 and 4.5 illustrates the process and the results, respectively.



FIGURE 4.4 – Performed test for latency.

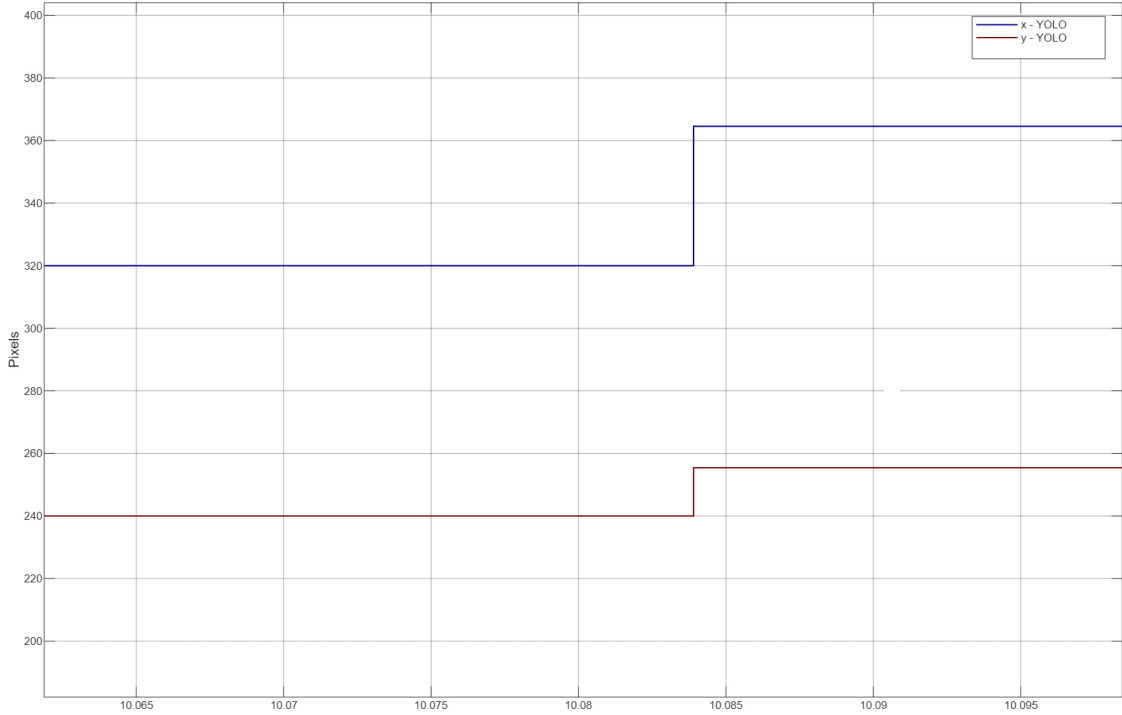


FIGURE 4.5 – Object recognition delay in seconds.

The results show that the system takes around 84 milliseconds to receive the object's position from the object detection algorithm. This time represents the sum of τ_{sensor} and τ_{network} . Although it is not possible to have a precise measurement of the simulated camera delay, YOLO's processing time is one of the outputs of the object detection algorithm. Since the average time spent by YOLO to process the image is 60 milliseconds, it is estimated that the UDP connection is spending around 24 milliseconds. Thus, $\tau_{\text{sensor}} = 60$ ms and $\tau_{\text{network}} = 24$ ms. Given that the Padé approximation is the sum $\tau_{\text{controller}}$ and τ_{actuator} , the application-perceived latency is approximately

$$\Delta\tau_{l,\text{application}} = 60 + 40 + 24 = 124 \text{ ms.} \quad (4.2)$$

4.2.2 Controlled Output

To understand how the system is behaving, we generate an input signal representing the angular displacement between the object's position and the principal axis, with reference in 0 (zero), and analyse the system's response. This method allows us to see the system's stability, overshoot and other metrics. Similarly to what was shown in Figure 4.4, we will create an object in the image after 10 seconds. The test performed and the system's response are shown in Figures 4.6 and 4.7.

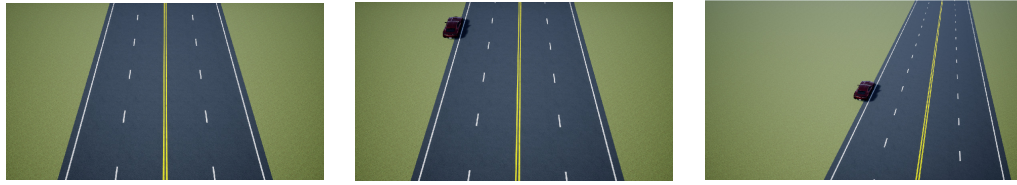


FIGURE 4.6 – Test performed for system behavior analysis.

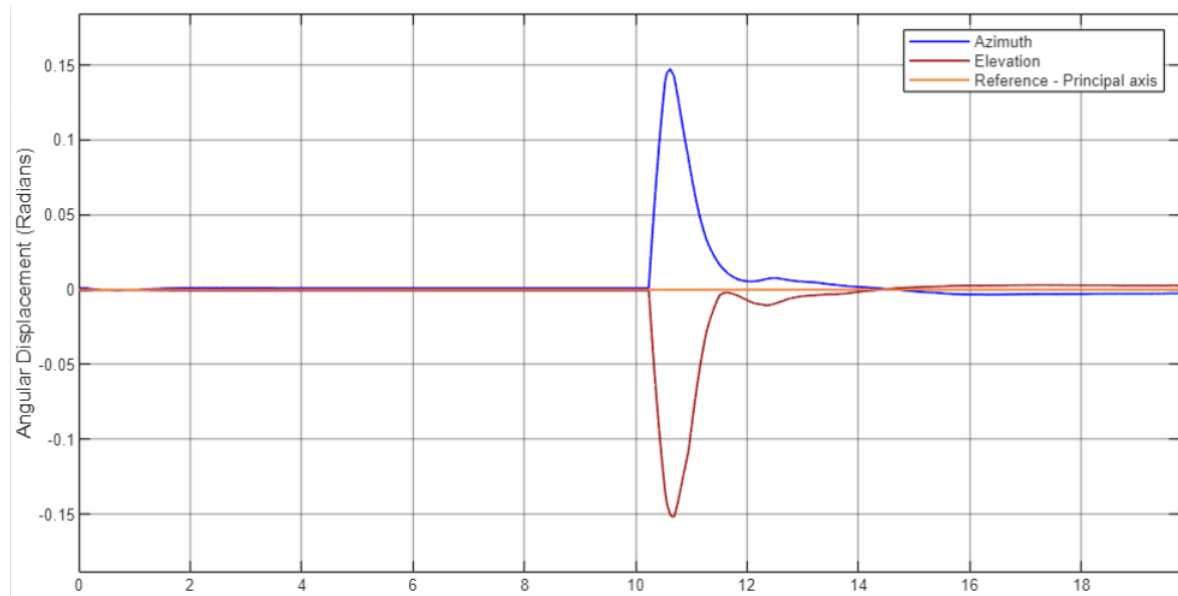


FIGURE 4.7 – System response to the input signal.

The result shows that the system is stable, although the latency in communication may affect the transient response. In 10 seconds, where the step occurs, it is possible to observe an overshoot for both axes, which leads to an oscillatory behavior that takes around 5 seconds to settle, entering in the steady-state. This shows opportunities for improvement in the system model and the PID controller.

5 Conclusions and Future Works

This chapter consolidates the main conclusions drawn from the development and validation of the image-based gimbal control system. It also outlines potential avenues for future research, aiming to further enhance the system's performance and applicability.

5.1 Conclusions

This Master's thesis successfully developed and validated a comprehensive control solution for real-time object tracking in images, utilizing a two-degrees-of-freedom (2-DOF) gimbal system. The work addressed the inherent challenges associated with complex gimbal dynamics, including nonlinearities and communication delays, by proposing an integrated architecture.

The core of the proposed solution involved the synergistic integration of a state-of-the-art object detection algorithm (YOLOv8), a state estimator (Kalman Filter) for signal smoothing, and a Proportional-Integral-Derivative (PID) control system. The control actuation was applied to a detailed Simulink model of a 2-DOF gimbal, responsible for adjusting the camera's orientation to maintain the object centered within the field of view.

Several key findings and achievements emerged from this study. A robust end-to-end system architecture was established, seamlessly connecting visual perception (YOLOv8), data communication (UDP protocol), state estimation (Kalman Filter), and motion control (PID controller) with the electromechanical dynamics of the gimbal. This integration was validated in a unified Simulink environment, demonstrating a fundamental step towards autonomous tracking systems. Regarding object detection performance, the YOLOv8 algorithm demonstrated its capability in real-time object detection, providing the necessary pixel coordinates for tracking. While general performance metrics (mAP@50 and mAP@50-95) showed promising trends during training, a class imbalance within the VisDrone2019 dataset was identified as a factor influencing detection accuracy across different object categories.

System latency management was a critical aspect, and the overall system latency,

encompassing camera delay, YOLO processing time, UDP communication, and the Controller and Actuator delays, resulted in approximately 124 milliseconds. The PID controller, designed using MATLAB's `pidtune()` function and refined through fine-tuning, demonstrated its ability to stabilize the gimbal's rotational axes. For the elevation axis, the step response showed stable behavior with an initial overshoot attributed to the transient effects of communication startup and noise. The system effectively tracked the zero reference (center of the image) with an approximate settling time of 5 seconds, exhibiting an underdamped response. The azimuth axis exhibited similar dynamic characteristics to the elevation axis, as expected due to the simplified model's assumptions of analogous dynamics. It achieved a lower settling time but presented a more oscillatory behavior, indicating areas for potential improvement. Finally, the gimbal model incorporated critical nonlinearities such as viscous and Coulomb frictions and considered the rotational inertia of the motor and encoder, enhancing the realism of the simulation environment. The inclusion of communication delay via Padé approximation further contributed to a more faithful evaluation under conditions emulating the physical world.

In conclusion, this work successfully demonstrated the viability of an image-based control system for a 2-DOF gimbal, providing a stable and responsive platform for object tracking. The integrated approach, coupled with careful modeling of system dynamics and real-time considerations, lays a solid foundation for future advancements in autonomous visual tracking systems.

5.2 Future Works

Building upon the achievements and insights gained from this master's thesis, several avenues for future research are identified to further enhance the performance, robustness, and real-world applicability of the image-based gimbal control system.

One area for future exploration involves advanced control strategies. This could include investigating the implementation of advanced nonlinear control techniques, such as Sliding Mode Control (SMC) (RAHMAT *et al.*, 2016), since this method is inherently better suited to handle the complex nonlinearities (e.g., Coulomb friction, actuator saturation, torque ripple) present in gimbal dynamics, potentially leading to improved transient response (reduced overshoot and oscillations) and enhanced disturbance rejection. Furthermore, adaptive control algorithms, such as Fuzzy logic controller (GALAL *et al.*, 2009), could be investigated to allow the controller to adjust its parameters in real-time in response to varying operating conditions or unmodeled dynamics, thereby increasing system robustness. Developing explicit compensation strategies for known disturbances, such as dynamic mass unbalance and torque ripple, could further improve tracking preci-

sion and reduce steady-state errors. Finally, implementing control strategies specifically designed to actively compensate for the measured communication and processing delays, rather than merely tolerating them, could involve predictive control elements or Smith predictors.

Another significant area for future work lies in object detection and state estimation enhancement. This includes addressing the identified dataset imbalance in YOLO training through resampling techniques or data augmentation strategies, and further optimizing YOLOv8 hyperparameters to improve detection accuracy across all classes, especially those with fewer instances. Exploring and comparing the performance of other state-of-the-art object detection algorithms (e.g., Faster R-CNN, SSD variants) for their suitability in real-time gimbal control applications, considering their trade-offs between accuracy and computational cost, is also a valuable direction. Additionally, extending the Kalman filter-based state estimator to track multiple objects simultaneously would require more sophisticated data association and state management techniques. Enhancing capabilities to estimate a target's distance and velocity, particularly when associated with ballistics, expands its applicability to military systems.

Gimbal model refinement presents further opportunities. This could involve incorporating more granular physical phenomena into the Simulink model, such as detailed bearing dynamics, structural flexibility of the gimbal arms, and more precise modeling of cross-coupling effects that were simplified in the current iteration. Integrating and compensating for the dynamic angular motion of the base (e.g., aircraft maneuvers), which was simplified by the fixed-base assumption in the current model, is critical for real-world applications.

Experimental validation is a crucial next step. This involves transitioning the validated control solution from the simulation environment to a physical 2-DOF gimbal prototype. This step is essential for real-world testing, allowing for the identification and mitigation of unmodeled dynamics and practical challenges not captured in simulation. Implementing Real-time Hardware-in-the-Loop (HIL) simulations to test the controller with actual hardware components (e.g., motors, encoders) connected to the simulated gimbal model would provide a more realistic validation environment before full physical deployment.

Finally, computational optimization is an important consideration. Optimizing the computational efficiency of the object detection and control algorithms for deployment on embedded systems or edge computing devices is typical in UAV applications.

These future works collectively aim to enhance the system's robustness, precision, and practical applicability, paving the way for more sophisticated and reliable image-based gimbal control systems in diverse engineering domains.

Bibliography

AL-E'MARI, S.; SANJALAWA, Y.; ALQUDAH, H. Integrating enhanced security protocols with moving object detection: A yolo-based approach for real-time surveillance. *In: 2nd International Conference on Cyber Resilience (ICCR). Proceedings [...]*. Dubai, United Arab Emirates: IEEE, 2024. p. 1–6.

BOIFFIER, J. **The Dynamics of Flight, The Equations**. Wiley, 1998. (The Dynamics of Flight). ISBN 9780471942375. Available at: <https://books.google.com.br/books?id=us8eAQAAIAAJ>.

BRUNTON, S. L.; KUTZ, J. N. **Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control**. 2. ed. Cambridge, USA: Cambridge University Press, 2022.

CUNHA, R.; MALACA, M.; SAMPAIO, V.; GUERREIRO, B.; NOUSI, P.; MADEMLIS, I.; TEFAS, A.; PITAS, I. Gimbal control for vision-based target tracking. *In: EURASIP European Signal Processing Conference (EUSIPCO). Proceedings [...]*. A Coruña, Spain: EUSIPCO, 2019.

DONE, G. Dynamics of flight: the equations. j-l. boiffier. john wiley and sons, baffins lane, chichester, west sussex p019 1ud, uk1998. 353 pp. illustrated. £34.95. **The Aeronautical Journal**, v. 103, n. 1025, p. 358–358, 1999.

EVERINGHAM, M.; GOOL, L. V.; WILLIAMS, C.; WINN, J.; ZISSERMAN, A. The pascal visual object classes (voc) challenge. **International Journal of Computer Vision**, v. 88, p. 303–338, June 2010.

GALAL, M.; MIKHAIL, N. G.; ELNASHAR, G. Fuzzy logic controller design for gun-turret system. *In: 13th International Conference on Aerospace Sciences and Aviation Technology (ASAT-13). Proceedings [...]*. Egypt: Military Technical College, 2009. Paper: ASAT-13-CT-34.

GATIMU, K.; DHAMODARAN, A.; JOHNSON, T. *et al.* Experimental study of qoe improvements towards adaptive hd video streaming using flexible dual tcp-udp streaming protocol. **Multimedia Systems**, v. 26, p. 479–493, 2020.

GIRSHICK, R. Fast r-cnn. *In: IEEE International Conference on Computer Vision (ICCV). Proceedings [...]*. Washington, DC: IEEE, 2015. p. 1440–1448.

GIRSHICK, R.; DONAHUE, J.; DARRELL, T.; MALIK, J. Rich feature hierarchies for accurate object detection and semantic segmentation. *In: IEEE Conference on*

- Computer Vision and Pattern Recognition. Proceedings** [...]. Columbus, Ohio: IEEE, 2014. p. 580–587.
- GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing (3rd Edition)**. Saddle River, New Jersey, USA: Prentice-Hall, Inc., 2006. ISBN 013168728X.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge, MA: MIT Press, 2016. <http://www.deeplearningbook.org>.
- GRANDINI, M.; BAGLI, E.; VISANI, G. **Metrics for Multi-Class Classification: an Overview**. 2020. Available at: <https://arxiv.org/abs/2008.05756>.
- HARTLEY, R. I.; ZISSERMAN, A. **Multiple View Geometry in Computer Vision**. Second. Cambridge, UK: Cambridge University Press, ISBN: 0521540518, 2004.
- HUBEL, D.; WIESEL, T. Receptive fields, binocular interaction, and functional architecture in the cat's visual cortex. **Journal of Physiology**, v. 160, p. 106–154, 1962.
- JIANG, S.; QIN, S.; PULSIPHER, J.; ZAVALA, V. Convolutional neural networks: Basic concepts and applications in manufacturing. October 2022.
- JOCHER, G.; CHAURASIA, A.; QIU, J. **Ultralytics YOLO**. January 2023. Available at: <https://github.com/ultralytics/ultralytics>.
- KIESEL, R.; JAKOB, F.; VOLLMER, T.; SCHMITT, R. H. Evaluation of ict for networked control systems of latency-critical applications in production. **Procedia CIRP**, v. 112, p. 238–243, 15th CIRP Conference on Intelligent Computation in ManufacturingEngineering, 14-16 July 2021, 2022. ISSN 2212-8271. Available at: <https://www.sciencedirect.com/science/article/pii/S2212827122012410>.
- KOLDA, T.; BADER, B. Tensor decompositions and applications. **SIAM Review**, v. 51, p. 455–500, August 2009.
- LACEY, T. Tutorial: the kalman filter 11.1 introduction 11.2 mean squared error. *In: . Proceedings* [...]. Cambridge, MA: MIT, 1998. Accessed on August 5, 2024. Available at: <https://api.semanticscholar.org/CorpusID:48574040>.
- LIN, T.-Y.; MAIRE, M.; BELONGIE, S.; BOURDEV, L.; GIRSHICK, R.; HAYS, J.; PERONA, P.; RAMANAN, D.; ZITNICK, C. L.; DOLLÁR, P. **Microsoft COCO: Common Objects in Context**. 2015. Available at: <https://arxiv.org/abs/1405.0312>.
- LIU, W.; ANGUELOV, D.; ERHAN, D.; SZEGEDY, C.; REED, S.; FU, C.-Y.; BERG, A. C. Ssd: Single shot multibox detector. *In: _____. Computer Vision - ECCV*. Amsterdam, NL: Springer International Publishing, 2016. p. 21–37. ISBN 9783319464480.
- MANE, S.; MANGALE, S. Moving object detection and tracking using convolutional neural networks. *In: Second International Conference on Intelligent Computing and Control Systems (ICICCS). Proceedings* [...]. Madurai, India: ICICCS, 2018. p. 1809–1813. June.
- MITCHELL, T. **Machine Learning**. McGraw-Hill, 1997. (McGraw-Hill International Editions). ISBN 9780071154673. Available at: <https://books.google.com.br/books?id=EoYBngEACAAJ>.

- MURPHY, K. P. **Machine Learning: A Probabilistic Perspective**. Massachusetts, USA: The MIT Press, 2012. ISBN 0262018020.
- PADILLA, R.; NETTO, S. L.; SILVA, E. A. B. da. A survey on performance metrics for object-detection algorithms. *In: International Conference on Systems, Signals and Image Processing (IWSSIP). Proceedings [...]*. Niterói, BR: IEEE, 2020. p. 237–242.
- POYRAZOĞLU, E. **Detailed modeling and control of a 2-DOF gimbal system**. Dissertation (Mestrado) — Middle East Technical University, 2017.
- RAHMAT, M. S.; HUDHA, K.; IDRIS, A. C.; AMER, N. H. Sliding mode control of target tracking system for two degree of freedom gun turret model. **Advances in Military Technology**, v. 11, n. 1, p. 13–28, 2016. Available at: <https://www.aimt.cz/index.php/aimt/article/view/1087>.
- RAMÍREZ, C. I. E. **Diseño e Implementación del Control de un Gimbal de Dos Grados de Libertad**. Dissertation (Mestrado) — Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional, 2016.
- REDMON, J.; DIVVALA, S.; GIRSHICK, R.; FARHADI, A. You only look once: Unified, real-time object detection. *In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Proceedings [...]*. Las Vegas, NV, USA: IEEE, 2016. p. 779–788.
- REN, S.; HE, K.; GIRSHICK, R.; SUN, J. Faster r-cnn: Towards real-time object detection with region proposal networks. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 39, n. 6, p. 1137–1149, 2017.
- RUE, A. K. Stabilization of precision electrooptical pointing and tracking systems. **IEEE Transactions on Aerospace and Electronic Systems**, AES-5, n. 5, p. 805–819, 1969.
- TONG, W.; XIANG, B.; WONG, W. Gimbal torque and coupling torque of six degrees of freedom magnetically suspended yaw gimbal. **International Journal of Mechanical Sciences**, ELSEVIER, UK, v. 168, p. 105312, 2020. ISSN 0020-7403. Available at: <https://www.sciencedirect.com/science/article/pii/S0020740319314456>.
- ULTRALYTICS. **Model Training with Ultralytics YOLO**. 2024. Accessed on June 10, 2024. Available at: <https://docs.ultralytics.com/modes/train/>.
- VINAYKUMAR, M.; JATOTH, R. K. Performance evaluation of alpha-beta and kalman filter for object tracking. *In: IEEE International Conference on Advanced Communications, Control and Computing Technologies. Proceedings [...]*. Ramanathapuram, India: IEEE, 2014. p. 1369–1373.
- YINGGE, H.; ALI, I.; LEE, K.-Y. Deep neural networks on chip - a survey. *In: . Proceedings [...]*. Busan, Korea (South): IEEE, 2020. p. 589–592.
- ZHOU, L.; CHENG, L.; PAN, C.; JIANG, Z. Generalized extended state observer based speed control for dc motor servo system. *In: . Proceedings [...]*. Wuhan, China: IEEE, 2018. p. 221–226.
- ZHU, P.; WEN, L.; DU, D.; BIAN, X.; FAN, H.; HU, Q.; LING, H. Detection and tracking meet drones challenge. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 44, n. 11, p. 7380–7399, November 2022.

FOLHA DE REGISTRO DO DOCUMENTO			
1. CLASSIFICAÇÃO/TIPO DM	2. DATA 03 de Setembro de 2024	3. DOCUMENTO Nº	4. Nº DE PÁGINAS 73
5. TÍTULO E SUBTÍTULO: Image-Based Gimbal Control Using a Convolutional Neural Network			
6. AUTOR(ES): Lucas Ávila Silva			
7. INSTITUIÇÃO(ÕES)/ÓRGÃO(S) INTERNO(S)/DIVISÃO(ÕES): Instituto Tecnológico de Aeronáutica – ITA			
8. PALAVRAS-CHAVE SUGERIDAS PELO AUTOR: Object Detection, Tracking, YOLO, Gimbal, Image			
9. PALAVRAS-CHAVE RESULTANTES DE INDEXAÇÃO:			
10. APRESENTAÇÃO: ITA, São José dos Campos. Curso de Mestrado. Programa de Pós-Graduação em Engenharia Aeronáutica			
11. RESUMO: A detecção de objetos em tempo real é fundamental para o avanço de sistemas autônomos e de segurança. A integração dessa tecnologia a gimbals multi-eixos não apenas proporciona estabilização de imagem e rastreamento preciso, mas também expande significativamente o campo de visão operacional. No contexto aeronáutico, essa integração se manifesta em sensores optoeletrônicos cruciais para operações militares e de reconhecimento. Contudo, um desafio persistente reside na manutenção do rastreamento confiável, dado o intrínseco equilíbrio entre a precisão do algoritmo de detecção e o custo computacional, bem como as dinâmicas complexas do gimbal (incluindo não linearidades e atrasos de comunicação). Este projeto de mestrado tem como objetivo principal desenvolver uma solução de controle robusta para aplicações de rastreamento de objetos em imagens em tempo real. Para tanto, emprega-se o YOLOv8, um algoritmo de detecção de última geração, cuja saída alimenta um sistema de controle PID. A atuação do controle é sobre um gimbal de dois graus de liberdade, responsável por ajustar a orientação de uma câmera para manter o objeto centralizado. A validação do desempenho das técnicas de controle propostas é realizada em um ambiente de simulação (Simulink), executado em tempo real, permitindo uma avaliação abrangente sob condições que emulam cenários operacionais.			
12. GRAU DE SIGILO: (X) OSTENSIVO () RESERVADO () SECRETO			